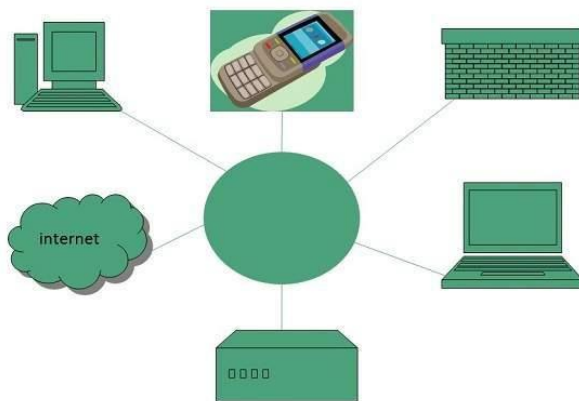


Chapter 1: The World Wide Web

1.1 Internet

The Internet is **a collection of computers connected by network cables or through satellite links**. Rather than connecting every computer on the Internet with every other computer, individual computers in an organization are normally connected in a local area network (LAN).

Internet is a system that interconnects the different computer systems across the world. It uses the Internet protocol suite to link devices located in different corners of the world. The Internet system carries an extensive range of information resources and services including World Wide Web (WWW), telephony, electronic mail, etc. It uses standard internet protocols, such as TCP/IP and HTTP, etc.



Some specific examples of how the Internet is used include: **Social media and content sharing**; E-mail and other forms of communication, such as Internet Relay Chat (IRC), Internet telephony, instant messaging, video conferencing.

Features of Internet

- Accessibility. An Internet is a global service and accessible to all. ...
- Easy to Use. ...
- Interaction with Other Media. ...
- Low Cost. ...
- Extension of Existing IT Technology. ...
- Flexibility of Communication. ...
- Security.

Internet Software

Internet Software comprises of all the tools needed for networking through computer. Following are a few important components of the Internet Software –

- Transmission Control Protocol/ Internet Protocol (TCP/IP)
- Internet Browser

Applications of internet

Internet applications are server-based applications. Following are a few Internet Applications –

- World Wide Web (WWW)
- Electronic mail (e-mail)
- File Transfer Protocol (FTP)
- Telnet (i.e., log-in to the computer located remotely)
- Internet Relay Chat (IRC) (Real time video chatting)

1.2 World Wide Web:

The World Wide Web—commonly referred to as WWW, W3, or the Web—is **an interconnected system of public webpages accessible through the Internet**. The Web is not the same as the Internet: the Web is one of many applications built on top of the Internet.

World Wide Web examples:

The World Wide Web is another way to describe the Internet, which is a network of computers which are connected and that share information and allow communication around the world. An example of the World Wide Web is **the Internet**. It's also known as the Web.

Web – Basic Elements:

On the simplest level, the Web physically consists of the following components –

- **Your personal computer** – This is the PC at which you sit to see the web.
- **A Web browser** – A software installed on your PC which helps you to browse the Web.
- **An internet connection** – This is provided by an ISP and connects you to the internet to reach to any Website.
- **A Web server** – This is the computer on which a website is hosted.
- **Routers & Switches** – They are the combination of software and hardware who take your request and pass to appropriate Web server.

The Web is known as a *client-server system*. Your computer is the client and the remote computers that store electronic files are the servers (where website is hosted).

How the Web Works:

When you enter something like Google.com the request goes to one of many special computers on the Internet known as *Domain Name Servers (DNS)*. All these requests are routed through various routers and switches. The domain name servers keep tables of machine names and their IP addresses, so when you type in Google.com it gets translated into a number, which identifies the computers that serve the Google Website to you.

When you want to view any page on the Web, you must initiate the activity by requesting a page using your browser. The browser asks a domain name server to translate the domain name you requested into an IP address. The browser then sends a request to that server for the page you want, using a standard called Hypertext Transfer Protocol or HTTP.

The server should constantly be connected to the Internet, ready to serve pages to visitors. When it receives a request, it looks for the requested document and returns it to the Web browser. When a request is made, the server usually logs the client's IP address, the document requested, and the date and time it was requested. This information varies server to server.

An average Web page actually requires the Web browser to request more than one file from the Web server and not just the HTML / XHTML page, but also any images, style sheets, and other resources used in the web page. Each of these files including the main page needs a URL to identify each item. Then each item is sent by the Web server to the Web browser and Web browser collects all this information and displays them in the form of Web page.

In Short,

We have seen how a Web client - server interaction happens. We can summarize these steps as follows –

A user enters a URL into a browser (for example, Google.com). This request is passed to a domain name server.

The domain name server returns an IP address for the server that hosts the Website (for example, 68.178.157.132).

The browser requests the page from the Web server using the IP address specified by the domain name server.

The Web server returns the page to the IP address specified by the browser requesting the page. The page may also contain links to other files on the same server, such as images, which the browser will also request.

The browser collects all the information and displays to your computer in the form of Web page.

1.3 Client-serverArchitecture

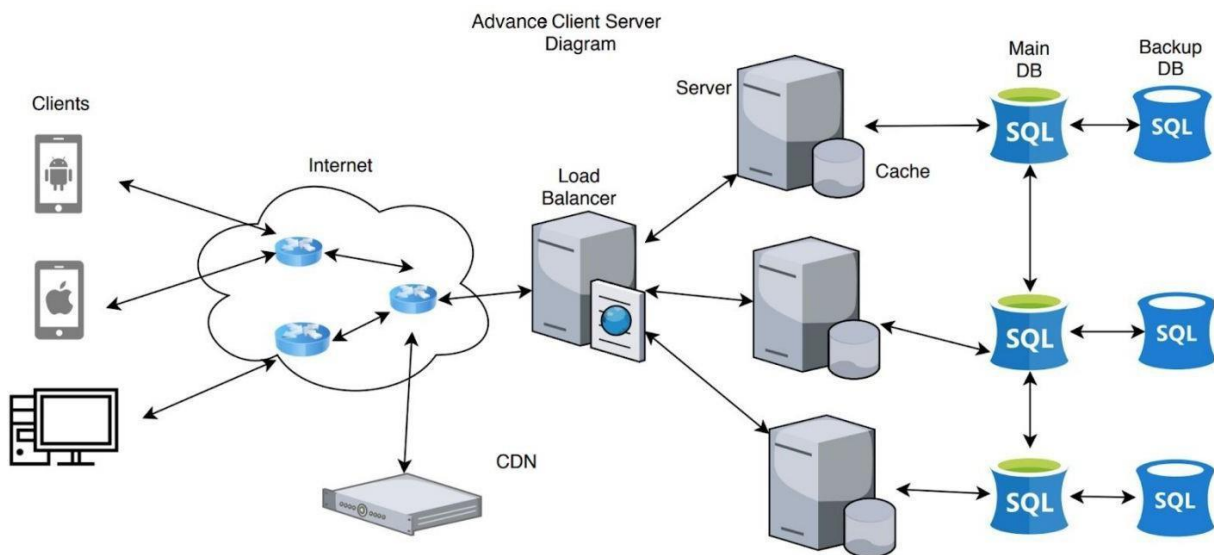
DefinitionClient-server architecture is a computing model in which the server hosts, delivers and manages most of the resources and services to be consumed by the client.

This type of architecture has one or more client computers connected to a central server over a network or internet connection. Client-server architecture is also known as a networking computing model or client-server network because all the requests and services are

delivered over a network.

- **Basic terminology:**

- Client: a piece of software or application that takes the input and sends request to the servers.
- Server: a piece of software that receives and processes requests from clients.
- Load-balancer: responsible for distributing incoming network traffic across a group of backend servers to optimize resource usage
- Network layer protocols such as TCP/IP



The flow of the data is unidirectional and forms a cycle. It is usually initiated by the client requesting some kind of data and the server processing the request and sending some kind of data back to the client via a protocol. Clients cannot directly talk to each other.

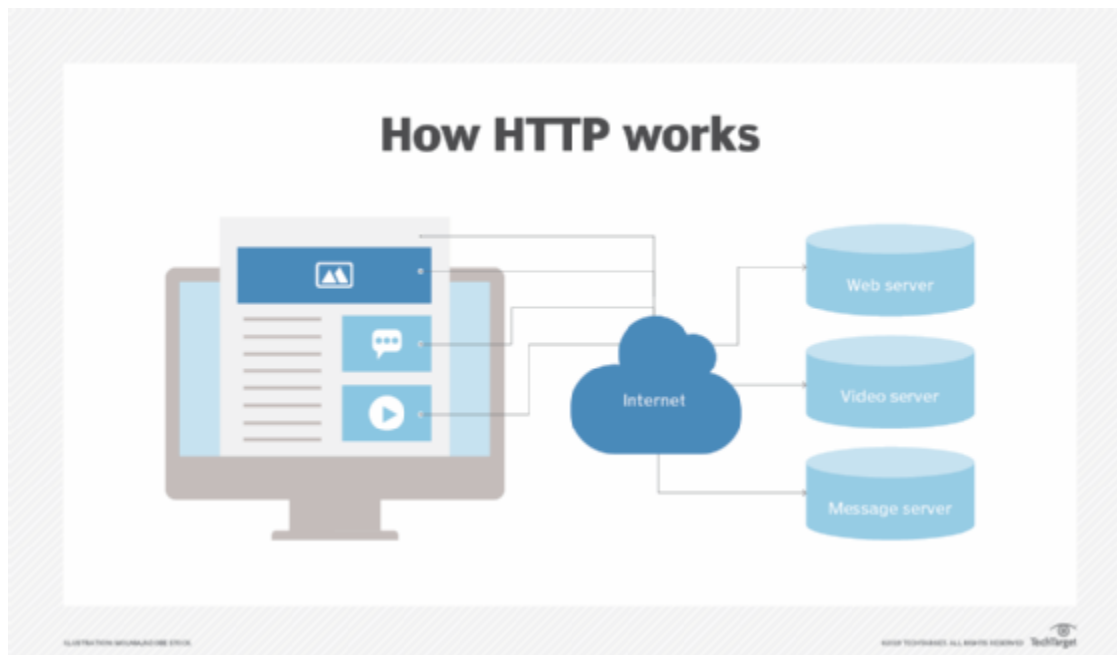
A typical topological data flow goes as follows:

1. Client requests data from server
2. Load balancer routes the request to the appropriate server
3. Server processes the request client
4. Server queries appropriate database for some data
5. Database returns the queried data back to the server
6. The server processes the data and sends the data back to the client

7. This process repeats.

Hypertext Transfer Protocol

HTTP (Hypertext Transfer Protocol) is the set of rules for transferring files -- such as text, images, sound, video and other multimedia files -- over the web. As soon as a user opens their web browser, they are indirectly using HTTP. HTTP is an application protocol that runs on top of the TCP/IP suite of protocols, which forms the foundation of the internet.



HTTP requests and responses

Each interaction between the client and server is called a **message**. HTTP messages are requests or responses. Client devices submit HTTP requests to servers, which reply by sending HTTP responses back to the clients.

HTTP requests.

This is when a client device, such as an internet browser, asks the server for the information needed to load the website. The request provides the server with the desired information it needs to tailor its response to the client device. Each HTTP request contains encoded data, with information such as:

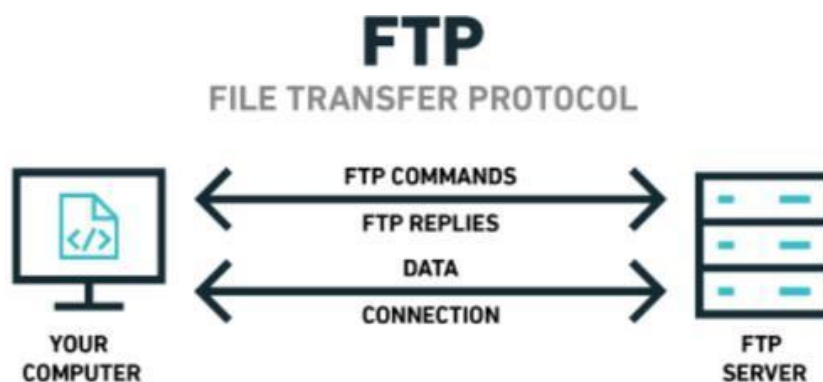
- **The specific version of HTTP followed.** HTTP and HTTP/2 are the two versions.
- **A URL.** This points to the resource on the web.
- **An HTTP method.** This indicates the specific action the request expects to receive from the server in its response.
- **HTTP request headers.** This includes data such as what type of browser is being used

and what data the request is seeking from the server. It can also include cookies, which show information previously sent from the server handling the request.

- **An HTTP body.** This is optional information the server needs from the request, such as user forms -- username/password logins, short responses and file uploads -- that are being submitted to the website.

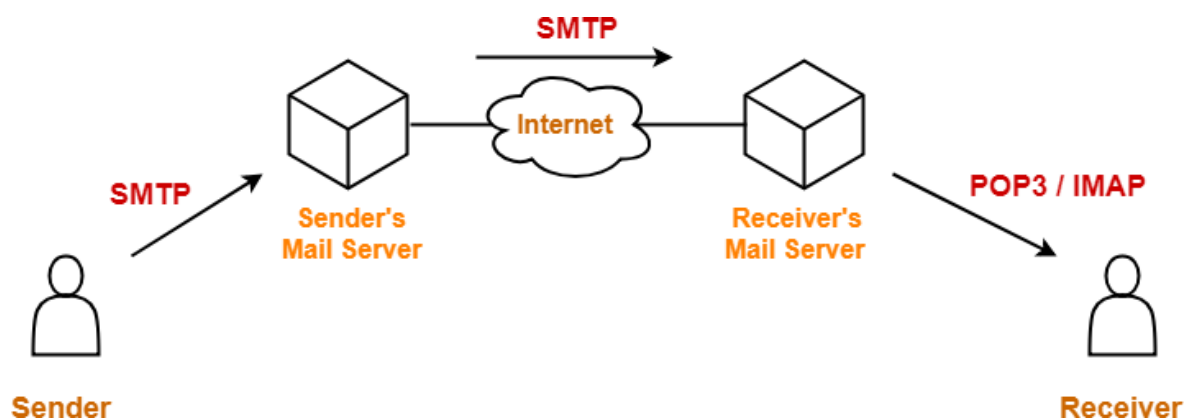
File Transfer Protocol

The **File Transfer Protocol (FTP)** is a standard communication protocol used for the transfer of computer files from a server to a client on a computer network. FTP is built on a client-server model architecture using separate control and data connections between the client and the server.^[1] FTP users may authenticate themselves with a clear-text sign-in protocol, normally in the form of a username and password, but can connect anonymously if the server is configured to allow it.



Simple Mail Transfer Protocol

The **Simple Mail Transfer Protocol (SMTP)** is an [internet standard communication protocol](#) for [electronic mail](#) transmission. Mail servers and other [message transfer agents](#) use SMTP to send and receive mail messages.



1.4 Web Design/web programming/Internet Programming

Web programming refers to the writing, markup and coding involved in Web development, which includes Web content, Web client and server scripting and network security. The most common languages used for Web programming are XML, HTML, JavaScript, Perl 5 and PHP. Web programming is different from just programming, which requires interdisciplinary knowledge on the application area, client and server scripting, and database technology

Web programming can be briefly categorized into client and server coding. The client side needs programming related to accessing data from users and providing information. It also needs to ensure there are enough plugins to enrich user experience in a graphic user interface, including security measures.

1. To improve user experience and related functionalities on the client side, JavaScript is usually used. It is an excellent client-side platform for designing and implementing Web applications.
2. HTML5 and CSS3 supports most of the client-side functionality provided by other application frameworks.

The server side needs programming mostly related to data retrieval, security and performance. Some of the tools used here include ASP, Lotus Notes, PHP, Java and MySQL. There are certain tools/platforms that aid in both client- and server-side programming. Some examples of these are Opa and Tersus.

5 Ways to Evaluate the Quality of Your Website Design

1. Strategy

Good website design is backed by strategy. Even the most attractive, user-friendly website isn't successful when it isn't achieving what your company needs. So ask yourself: Do new visitors get a clear sense of who you are and what you offer when they arrive at your site? Does your design direct visitors to do what you want them to do? Is there clear strategy informing your design? If not, your design is not as good as it could be. To evaluate the effectiveness of strategy in your website design, run it through this checklist of questions:

- What category is my business, and is that obvious on my website?
- What is the purpose of this website, and is the design accomplishing it?
- Who is my target audience, and how does the design take them into consideration?
- What do I want my audience to do, and is the design encouraging that action?

2. Usability

Usability is all about the practical considerations of what goes into good website design, such as speed, user-friendliness, security, technical details like sitemaps, etc. A lot of these details aren't visually apparent; you don't see a website's security when you type in its URL. Nonetheless, usability is a make-or-break issue for websites that work. If a visitor can't

find what he or she is looking for because of poor navigation, he or she will usually leave. If pages take too long to load, both search engines and visitors will notice.

3. Style

Beauty may be relative, but that doesn't mean there aren't clear aesthetic principles to guide your website design. The best designs will align with their brands, create positive impressions for visitors, be clean, and complement the content they're communicating. To test the effectiveness of your website's aesthetic, consider the following:

- Does my website's style align with my brand in terms of colors, feel, graphics, etc?
- Is the style consistent throughout the website?
- Will the style suit my target audience? (i.e., cartoons on a toy company website, elegant layout on a legal website)
- What feel does the website give—Orderly or messy? Sparse or crowded? Playful or formal?—and how does that align with my goals?
- Where are photos or decorative touches getting in the way of my message?

4. Content

The two main considerations regarding content are readability and usefulness. Readability is important because if your visitors can't make out your content, whether that's because it's too small or in a pale color or in an unreadable font, there's no way for your message to get across. Usefulness is just as important, however, because if your content doesn't matter to your reader, you lose him or her anyway. Here are some questions to run your website content through to evaluate its quality:

- Are the fonts I've chosen readable?
- Is there enough contrast between background colors and font colors?
- Is all the text big enough?
- Will this content be relevant to the reader?
- Is the content concise but still useful?

5. Search Optimization

There are many ways that the design of your website impacts search optimization—as **Search Engine Journal points out**, –Search engine optimization and social networking all start with strong website design.¶ Does your website have a lot of graphics, for example? If so, search engines can't see them. You need to add ALT tags to your image descriptions in order for search engines to know what you're showing. Is your HTML efficient? If not, it could hurt your search rankings. Ask yourself the following questions to ensure **optimized**

website design:

- Are all my images optimized with ALT tags?
- Is my coding efficient, or are there extraneous lines that could be eliminated?
- Have I used relevant keywords in title tags, meta descriptions, heading tags, etc.?
- Do I have a sitemap?

1.5 Web Hosting

Website hosting/Web hosting/web shoting is an online service that enables you to publish your website or web application on the internet. So, anyone who has access to the internet has access to your website. In practice, it usually refers to the service you get from a web hosting provider like one.com. When you sign up for a web hosting service, you basically rent some space on a physical server where you can store all the files and data necessary for your website to work properly.

The **server** that hosts your website is a physical computer that runs continuously to make the site available for visitors all the time and is called web server. **Buying servers for web hosting** will allow you to store all the data of your website in the servers of your provider. Once a user enters your domain name into their browser's address bar, the web host's server will transfer all the files necessary to load your website.

You can host a website yourself, but it requires extensive technical skills for preparing a server for hosting. Self-hosting entails setting up and configuring a web server from scratch, including the equipment, infrastructure, hardware, and software. Furthermore, you will also have to handle all the ongoing maintenance.

A web hosting service provider ensures that your website performs optimally and with better security protocols. In addition, it simplifies the many complex aspects of hosting a website – from software installation to technical support.

We can host a website but hosting we should be aware of following things is detail for the successful web hosting(Requirements of hosting)

Requirements for website hosting.

1. **Setup of web server:** You should know how to setup a WWW server software on your computer. This is a software that allows Internet users to access the web files on your computer.
 - ✓ On the hardware side, a web server is a computer that stores web server software and a website's component files (for example, HTML documents, images, CSS stylesheets, and JavaScript files). A web server connects to the Internet and supports physical data interchange with other devices connected to the web.
 - ✓ On the software side, a web server includes several parts that control how web users access hosted files. At a minimum, this is an *HTTP server*. An HTTP server is software that understands **URLs** (web addresses) and **HTTP** (the protocol your browser uses to view webpages). An HTTP server can be accessed

through the domain names of the websites it stores, and it delivers the content of these hosted websites to the end user's device

2. **Availability of server:** Your computer has to stay on at all times. Each time you turn it off or reboot, your website will become unavailable. An available website is a website that is accessible and usable as expected by the user. When speaking about availability, we often refer to the ratio of the available time to the total time. Typically, a site's availability is described as a percentage such as 99.9% availability. For a site to have 100% uptime is an elusive, if not impossible goal over the long term. Most providers often have an availability goal they try to reach and maintain. Let's take a look at how to calculate the uptime percentage.
3. **Domain name and registration.** Domain names are alphanumeric aliases used to access websites; for example, Google's domain name is 'google.com' and their IP address is 192.168.1.1. For hosting you need a domain name, for getting domain name we have to register a domain name with domain name registrar.
4. **FTP Support:** The use of FTP (file transfer protocol) lets you upload files from your local computer to your web server. If you build your website using your own HTML files, you can transfer the files from your computer to the web server through FTP, allowing your website to be accessed through the internet.
5. **Adequate Storage Space:** What's adequate can vary from site to site, but if you find a hosting provider who can add space when you need it, you should be fine. if you're building your website on your hard drive or another device, check how much space the site uses, and make sure to have more than that amount of space in your web hosting package. You'll need enough space to hold all of your images, videos, music, and any other content for your site.
6. **Reasonable Bandwidth :** Your Internet service provider supports you running websites on your home computer. You will need to investigate your bandwidth quota. Your website will load slowly as home internet connections are not designed for serving web pages. Bandwidth is the maximum measure of data that can transfer to and from your website over a specific amount of time. This data includes uploads, downloads, and website visits. Websites with a lot of content and high traffic require the most bandwidth. Websites with low-to-moderate content and traffic don't require as much bandwidth. You want to be sure that the web hosting company can accommodate the amount of bandwidth you need. If not, your site will slowly load and operate.
7. **Strong Backup Plan :** Without a backup plan, your website can be destroyed by hackers, a fire at the server's location, or human error. Anything can happen. You need to know that your website is being regularly backed up to an offsite server or servers.
8. **Strong Security:** The best web hosts provide strong security to keep hackers, malware, and SPAM from invading your site. For eCommerce sites, even more security is required. A web host should offer a minimum of a TLS/SSL certificate, spam protection, and site backups. For e-commerce sites, the web host should offer PCI compliance. Only PCI-compliant websites can accept payments online. A TLS/SSL certificate means that all data traveling to and from your website is encrypted (scrambled). Everything such as credit card information and passwords is encrypted and safe from thieves.

9. **Scalability:** Scalability describes a system's elasticity. Application scalability is the ability of an application to handle a growing number of users and load, without compromising on performance and causing disruptions to user experience.
10. **Support:** While developing and managing your website, you may face issues you cannot solve. In those cases, an excellent technical support team is essential. The best web hosting providers provide 24/7 customer support through multiple channels, such as: Knowledge bases, Forums, Live chat, Chatbots, Email support etc

Web hosts provide the [hosting technology](#) and resources required for the effective and secure operation of your website. They are responsible for keeping the server up and running, implementing security measures, and ensuring that data such as texts, photos, and other files are transferred successfully to the visitors' browsers. The web hosting service/service provider should provide all above features and requirements for web hosting. Some example of web hosting service are IONOS,,HostGator, Bluehost, DreamHost, MochaHost, GoDaddy etc

Domain Name Registration

Domain name registrar is a business that handles the reservation of domain names as well as the assignment of [IP addresses](#) for those [domain names](#). Domain names are alphanumeric aliases used to access websites; for example, Google's domain name is 'google.com' and their IP address is 192.168.1.1. Domain names make it easier to access websites without having to memorize and enter numeric IP addresses. (For more on how this works, see [What Is DNS?](#))

A domain name is a string of text that maps to a numeric [IP address](#), used to access a website from client software. In plain English, a domain name is the text that a user types into a browser window to reach a particular website. For instance, the domain name for Google is 'google.com'.

The actual address of a website is a complex numerical IP address (e.g. 103.21.244.0), but thanks to [DNS](#), users are able to enter human-friendly domain names and be routed to the websites they are looking for. This process is known as a DNS lookup.

1. Find a domain name registrar.
2. Search for your domain name.
3. Finalize your domain name choice.
4. Choose a domain name suffix, such as .com or .net.
5. Purchase the domain name.
6. Add Domain ID protection.

1. Find a domain name registrar.

The non-profit [Internet Corporation for Assigned Names and Numbers \(ICANN\)](#) oversees the entire system of domain names, and it allows outside companies, called domain registrars, to sell and manage domain names. You will need to register your domain name through a registrar.

Some of the most popular domain registrars include [GoDaddy](#), [Bluehost](#), and [Domain.com](#), although there are many more to choose from. Be sure to carefully consider pricing and policies for each one before [choosing your domain registrar](#), as it will be managing your domain name.

2. Search for your domain name's availability.

Once you have found the right domain registrar for you, you'll need to search for your domain name using the registrar's search bar. There are millions of domain names out there, with thousands more added daily.

If you have your heart set on a domain name before doing a search, you might be disappointed to find it is already taken. Keep an open mind and incorporate important keywords into your domain when appropriate.

3. Finalize your domain name choice.

Once you've brainstormed several domain options, consider which ones are available and choose the one that fits your brand best and will also be easy for users to find.

4. Choose a domain name suffix, such as .com or .net.

After settling on the domain name, consider the suffix. The most popular suffix is .com, although .net and .org are also popular. These are considered to be top-level domains, or the highest-level domain suffixes in the domain naming system.

There are also other varieties to consider, such as country code top-level domains. These domain names end in a suffix that is particular to a specific country. A website in Germany might end in .de, for example.

Finally, there is another class of domain name suffixes called "sponsored, top-level domains," which are sponsored by a specific community related to the domain name. For example, .gov is for the U.S. government and .edu is for education organizations.

For most websites, .com is the best suffix for its ease of use, but if you have a specialized website, you might consider a country code or sponsored top-level domain.

5. Purchase the domain name.

When you have settled on the domain name and a suffix, you will pay to register the domain name with the domain registrar. This is not a one-time purchase, however. Typically, you will pay to own the domain name for one year, after which you can renew your registration for a fee. You can expect a registration fee of about \$10 to \$15.

The screenshot shows a domain registration interface. At the top, it says "Your Groovy Site" with the URL "yourgroovydomain.com". Below this, the domain "yourgroovydomain.com" is listed with a "Domain Registration" label and a price of "\$18". A "Renew Now" button is visible. A text block explains that the domain replaces the free WordPress address "yourgroovysite.wordpress.com". Below this, there are links for "Domain Settings" and "Domain Registration Agreement". At the bottom, a table provides details about the registration:

Owner	Price	Domain renews on	Payment method
You	\$18 USD / year	September 13, 2019	1111

An orange arrow points to the "Domain renews on" date in the table.

[Image Source](#)

6. Add domain ID protection.

When you register a domain name with ICANN, you must provide your contact information including your name, phone number, physical address, and email address.

As soon as your domain name is registered, this contact information becomes available to the public — unless you pay for domain privacy through your domain registrar. This domain privacy will shield your information from view, keeping your personal information safe from spammers or worse, identity thieves.

A graphic titled "Domain WHOIS Privacy" with a red padlock icon. It explains that domain registration information is public by default and that HostUpon offers Whois Domain Privacy from PrivacyProtect.org to protect this information. It compares the registrant details visible without privacy (John Smith, 123 Your Street, etc.) versus with privacy (PrivacyProtect.org, PO Box 16, etc.).

Domain WHOIS Privacy

Did you know that when you register a domain, your personal information is available to everyone? This includes your name, address, phone number and email.

HostUpon offers Whois **Domain Privacy** from PrivacyProtect.org so that your personal information is protected if someone does a whois search on your domain. This means no solicitation from unwanted companies and spambots.

Without Domain Privacy	With Domain Privacy
Registrant:	Registrant:
John Smith	PrivacyProtect.org
123 Your Street	PO Box 16
Your City, Your State 12345	Queensland, QLD 4218
Your Country	Australia
555-555-5555	45+36946676

[Image Source](#)

Once you've got the domain name in hand, you're ready to find [hosting](#) and [build your website](#). Consider a [free CMS](#), [like HubSpot](#), to design your website, strategize for SEO, and create a blog.

Cost to Register a Domain Name

According to [HostGator](#), domain name cost ranges from free to tens of thousands of dollars, with the average domain name settling around \$10-\$12 annually. These factors

HTML Introduction

- HTML stands for Hyper Text Markup Language
- HTML is the standard markup language for creating Web pages
- HTML describes the structure of a Web page
- HTML consists of a series of elements
- HTML elements tell the browser how to display the content
- HTML elements label pieces of content such as "this is a heading", "this is a paragraph", "this is a link", etc.

HTML stands for Hyper Text Markup Language. An HTML file is a text file containing markup tags. The markup tags tell the Web browser how to display the page. An HTML file must have an 'htm' or 'html' file extension. An HTML file can be created using a simple text editor. The rule-making body of the Web is World Wide Web Consortium (W3C). W3C puts together specifications for Web standards. The most essential Web standards are HTML, CSS and XML. The latest HTML standard is XHTML 1.0

Markup Language: Markup language refers to a text-encoding system consisting of a set of symbols inserted in a text document to control its structure, formatting, or the relationship between its parts. Markup is often used to control the display of the document or to enrich its content to facilitating automated processing.

Creating a simple web page

1. Start Notepad.
2. Type in the

```
<html>
<head>
<title>Title of page</title>
</head>
<body>
This is a very basic webpage. <b>This text will be displayed in bold</b>
</body>
</html>
```

3. Save the file as "firstpage.html".
4. Double click the saved file the browser will display the page.

HTM or HTML Extension?

When you save an HTML file, you can use either the .htm or the .html extension. We have used **.html** in our example.

HTML Tags

1. HTML tags are used to mark-up HTML elements
2. HTML tags are surrounded by the two characters < and >
3. The surrounding characters are called angle brackets

4. HTML tags normally come in pairs like *<name of tag>* content of tag *</name of tag>* for example to write bold letters `<b>` bold letters `</b>`
5. The first tag in a pair is the start tag, the second tag is the end tag
6. The text between the start and end tags is the element content
7. HTML tags are not case sensitive, `<b>` means the same as `<B>`

Use Lowercase Tags?

We have just said that HTML tags are not case sensitive: `<B>` means the same as `<b>`. It is recommended to always use small case because If you want to prepare yourself for the next generations of HTML, you should start using lowercase tags. The World Wide Web Consortium recommends lowercase tags in their HTML 4 recommendation, and XHTML (the next generation HTML) demands lowercase tags.

HTML ELEMENTS

An HTML element is defined by a start tag, some content, and an end tag:

`<tagname>` Content goes here... `</tagname>`

The HTML **element** is everything from the start tag to the end tag:

`<h1>`My First Heading`</h1>`

`<p>`My first paragraph.`</p>`

Start tag	Element content	End tag
<code><h1></code>	My First Heading	<code></h1></code>
<code><p></code>	My first paragraph.	<code></p></code>

TAG Attributes/ HTML Attributes: Tags can have attributes. Attributes can provide additional information about the HTML elements on your page. This tag defines the body element of your HTML page: `<body>`. With an added bgcolor attribute, you can tell the browser that the background color of your page should be red, like this:

`<body bgcolor="red">`.

Attributes always come in name/value pairs like this: `name="value"`. Attributes are always added to the start tag of an HTML element. In above example we have used bgcolor attribute for HTML Body Tag (start tag), it will change background color of body.

Quote Styles: "red" or 'red'?

Attribute values should always be enclosed in quotes. Double style quotes are the most common, but single style quotes are also allowed. In some rare situations, like when the attribute value itself contains quotes, it is necessary to use single quotes: for attributes also we

can use both upper and lower case but The World Wide Web Consortium recommends lowercase for writing attributes.

Comments in HTML The comment tag is used to insert a comment in the HTML source code. A comment will be ignored by the browser. You can use comments to explain your code, which can help you when you edit the source code at a later date.

```
<!-- This is a comment -->
```

Note: that you need an exclamation point after the opening bracket, but not before the closing bracket.

A Simple HTML Document

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>My First Heading</h1>
<p>My first paragraph.</p>

</body>
</html>
```

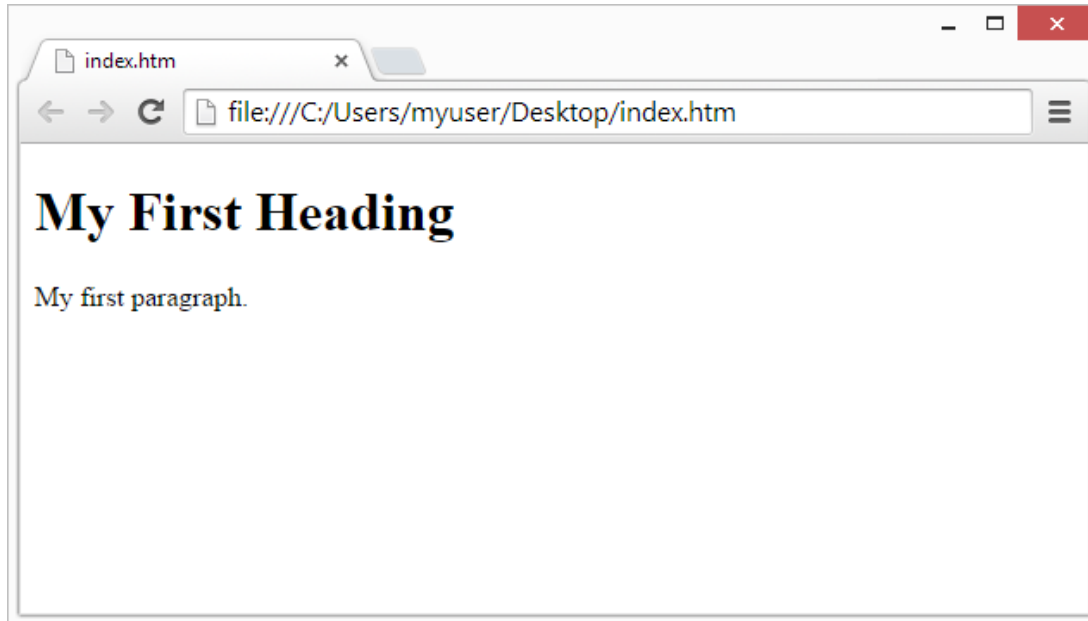
Example Explained

- The `<!DOCTYPE html>` declaration defines that this document is an HTML5 document
- The `<html>` element is the root element of an HTML page
- The `<head>` element contains meta information about the HTML page
- The `<title>` element specifies a title for the HTML page (which is shown in the browser's title bar or in the page's tab)
- The `<body>` element defines the document's body, and is a container for all the visible contents, such as headings, paragraphs, images, hyperlinks, tables, lists, etc.
- The `<h1>` element defines a large heading
- The `<p>` element defines a paragraph

Web Browsers

The purpose of a web browser (Chrome, Edge, Firefox, Safari) is to read HTML documents and display them correctly.

A browser does not display the HTML tags, but uses them to determine how to display the document:



HTML Page Structure

Below is a visualization of an HTML page structure: it contains

1. Document type declaration:

The `<!DOCTYPE>` declaration represents the document type, and helps browsers to display web pages correctly.

It must only appear once, at the top of the page (before any HTML tags).

The `<!DOCTYPE>` declaration is not case sensitive.

The `<!DOCTYPE>` declaration for HTML5 is:

`<!DOCTYPE html>`

2. html TAG: HTML tag The `<html>` tag represents the root of an HTML document.

The `<html>` tag is the container for all other HTML elements (except for the `<!DOCTYPE>` tag). Note: You should always include the lang attribute inside the `<html>` tag, to declare the language of the Web page. We use it as first tag because it helps browsers recognise that a given document is a HTML document

3. header information (`<head>` tag): The text between the `<head>` tag and the `</head>` tag is header information. The `<head>` element is a container for metadata (data about data) and is placed between the `<html>` tag and the `<body>` tag. Metadata is data about the HTML. Header information is not displayed in the browser window like title of page.

4. Body of html document (`<body>` Tag):

- The **<body>** tag defines the document's body.
- HTML **<body>** tag defines the main content of an HTML document which displays on the browser.
- The **<body>** must be the second element after the **<head>** tag or it should be placed between **</head>** and **</html>** tags.
- The **<body>** element contains all the contents of an HTML document, such as headings, paragraphs, images, hyperlinks, tables, lists, etc.
- **Note:** There can only be one **<body>** element in an HTML document.

```
<html>
<head>
<title>Page title</title>
</head>

<body>
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
<p>This is another paragraph.</p>
</body>

</html>
```

Types of tags: HTML contains dozens of tags, but the normally usefull tags can be classified in the following categories

1. **HTML Meta tags:** the tags which defines information about page, of define metadata about the page are fall down in this category.examples : DOCTYPE, title, link, meta etc....
2. **Text tags:** the tags related with formatting of text in html document. Like **<p>** for paragraph and heading tags.
3. **Link :** To define the link in html page, it may internal or external link.
4. **Image and object tags:** The tags which are used to insert image and other objects in the html document. Example ****
5. **List :** To define ordered and unordered list in html documents.
6. **Table .** Used to inser tables in the html document and its formatting like inserting row and coloumn, alignment, cell spacing and padding etc...
7. **Form:** To insert forms in html page for taking input from user.
8. **Scripting:** for using scripting language programming like server or client side programming,

Various html tags and there attributes:

HTML TAGS

page 1 of 2

Basic Tags

<code><html> </html></code>	Creates an HTML document
<code><head> </head></code>	Sets off the title & other info that isn't displayed
<code><body> </body></code>	Sets off the visible portion of the document
<code><title> </title></code>	Puts name of the document in the title bar; when bookmarking pages, this is what is bookmarked

Body attributes (only used in email newsletters)

<code><body bgcolor=?></code>	Sets background color, using name or hex value
<code><body text=?></code>	Sets text color, using name or hex value
<code><body link=?></code>	Sets color of links, using name or hex value
<code><body vlink=?></code>	Sets color of visited links, using name or hex value
<code><body alink=?></code>	Sets color of active links (while mouse-clicking)

Text Tags

<code><pre> </pre></code>	Creates preformatted text
<code><h1> </h1> --> <h6> </h6></code>	Creates headlines -- H1=largest, H6=smallest
<code> </code>	Creates bold text (should use <code></code> instead)
<code><i> </i></code>	Creates italicized text (should use <code></code> instead)
<code><tt> </tt></code>	Creates typewriter-style text
<code><code> </code></code>	Used to define source code, usually monospace
<code><cite> </cite></code>	Creates a citation, usually processed in italics
<code><address> </address></code>	Creates address section, usually processed in italics
<code> </code>	Emphasizes a word (usually processed in italics)
<code> </code>	Emphasizes a word (usually processed in bold)
<code> </code>	Sets size of font - 1 to 7 (should use CSS instead)
<code> </code>	Sets font color (should use CSS instead)
<code> </code>	Defines the font used (should use CSS instead)

Links

<code>clickable text</code>	Creates a hyperlink to a Uniform Resource Locator
<code>clickable text</code>	Creates a hyperlink to an email address
<code></code>	Creates a target location within a document
<code>clickable text</code>	Creates a link to that target location

Formatting

<code><p> </p></code>	Creates a new paragraph
<code>
</code>	Inserts a line break (carriage return)
<code><blockquote> </blockquote></code>	Puts content in a quote - indents text from both sides
<code><div> </div></code>	Used to format block content with CSS
<code> </code>	Used to format inline content with CSS

Lists

<code> </code>	Creates an unordered list
<code><ol start=?> </code>	Creates an ordered list (start=xx, where xx is a counting number)
<code> </code>	Encompasses each list item
<code><dl> </dl></code>	Creates a definition list
<code><dt></code>	Precedes each definition term
<code><dd></code>	Precedes each definition

Graphical elements

<code><hr></code>	Inserts a horizontal rule
<code><hr size=?></code>	Sets size (height) of horizontal rule
<code><hr width=?></code>	Sets width of rule (as a % or absolute pixel length)
<code><hr noshade></code>	Creates a horizontal rule without a shadow
<code></code>	Adds image; it is a separate file located at the URL
<code></code>	Aligns image left/right/center/bottom/top/middle (use CSS)
<code></code>	Sets size of border surrounding image (use CSS)
<code></code>	Sets height of image, in pixels
<code></code>	Sets width of image, in pixels
<code></code>	Sets the alternate text for browsers that can't process images (required by the ADA)

Forms

<code><form> </form></code>	Defines a form
<code><select multiple name=? size=?> </select></code>	Creates a scrolling menu. Size sets the number of menu items visible before user needs to scroll.
<code><select name=?> </select></code>	Creates a pulldown menu
<code><option></code>	Sets off each menu item
<code><textarea name=? cols="x" rows="y"> </textarea></code>	Creates a text box area. Columns set the width; rows set the height.
<code><input type="checkbox" name=? value=?></code>	Creates a checkbox.
<code><input type="checkbox" name=? value=? checked></code>	Creates a checkbox which is pre-checked.
<code><input type="radio" name=? value=?></code>	Creates a radio button.
<code><input type="radio" name=? value=? checked></code>	Creates a radio button which is pre-checked.
<code><input type="text" name=? size=?></code>	Creates a one-line text area. Size sets length, in characters.
<code><input type="submit" value=?></code>	Creates a submit button. Value sets the text in the submit button.
<code><input type="image" name=? src=? border=? alt=?></code>	Creates a submit button using an image.
<code><input type="reset"></code>	Creates a reset button

Tables (use only for data layout - use CSS for page layout)

<code><table> </table></code>	Creates a table
<code><tr> </tr></code>	Sets off each row in a table
<code><td> </td></code>	Sets off each cell in a row
<code><th> </th></code>	Sets off the table header (a normal cell with bold, centered text)

HTML5 input tag attributes

(not all browsers support; visit <http://caniuse.com> for details)

<code><input type="email" name=?></code>	Sets a single-line textbox for email addresses
<code><input type="url" name=?></code>	Sets a single-line textbox for URLs
<code><input type="number" name=?></code>	Sets a single-line textbox for a number
<code><input type="range" name=?></code>	Sets a single-line text box for a range of numbers
<code><input type="date/month/week/time" name=?></code>	Sets a single-line text box with a calendar showing the date/month/week/time
<code><input type="search" name=?></code>	Sets a single-line text box for searching
<code><input type="color" name=?></code>	Sets a single-line text box for picking a color

Table attributes (only use for email newsletters)

<code><table border=?></code>	Sets the width of the border around table cells
<code><table cellpadding=?></code>	Sets amount of space between table cells
<code><table cellspacing=?></code>	Sets amount of space between a cell's border and its contents
<code><table width=?></code>	Sets width of the table in pixels or as a percentage
<code><tr align=?></code>	Sets alignment for cells within the row (left/center/right)
<code><td align=?></code>	Sets alignment for cells (left/center/right)
<code><tr valign=?></code>	Sets vertical alignment for cells within the row (top/middle/bottom)
<code><td valign=?></code>	Sets vertical alignment for cell (top/middle/bottom)
<code><td rowspan=?></code>	Sets number of rows a cell should span (default=1)
<code><td colspan=?></code>	Sets number of columns a cell should span
<code><td nowrap></code>	Prevents lines within a cell from being broken to fit

HTML Colors : Colors are very important to give a good look and feel to your website. You can specify colors on page level using `<body>` tag or you can set colors for individual tags using **bgcolor** attribute.

The `<body>` tag has following attributes which can be used to set different colors –

- **bgcolor** – sets a color for the background of the page.
- **text** – sets a color for the body text.
- **alink** – sets a color for active links or selected links.
- **link** – sets a color for linked text.
- **vlink** – sets a color for *visited links* – that is, for linked text that you have already clicked on.

HTML Color Coding Methods

There are following three different methods to set colors in your web page – 1. Color names 2. Hex codes 3. color decimal/percentage value in RGB format

Color names – You can specify direct a color name to set text or background color. W3C has listed 16 basic color names that will validate with an HTML validator but there are over 200 different color names supported by major browsers.

Here is the list of W3C Standard 16 Colors names and it is recommended to use them.

	Black		Gray		Silver		White
	Yellow		Lime		Aqua		Fuchsia
	Red		Green		Blue		Purple
	Maroon		Olive		Navy		Teal

Hex codes – A six-digit code representing the amount of red, green, and blue that makes up the color. It start with # then. The first two digits(RR) represent a red value, the next two are a green value(GG), and the last are the blue value(BB). Each value for R,G and B can be 00 to ff. Some examples are given below

Color	Color HEX
	#000000
	#FF0000
	#00FF00
	#0000FF
	#FFFF00
	#00FFFF
	#FF00FF
	#C0C0C0
	#FFFFFF

Color decimal or percentage values – This color value is specified using the **rgb()** property. This property takes three values, one each for red, green, and blue. The value can be an integer between 0 and 255 or a percentage. **Note** – All the browsers does not support **rgb()** property of color so it is recommended not to use it. Following is a list to show few colors using RGB values.

Color	Color RGB
	rgb(0,0,0)
	rgb(255,0,0)
	rgb(0,255,0)
	rgb(0,0,255)
	rgb(255,255,0)
	rgb(0,255,255)
	rgb(255,0,255)
	rgb(192,192,192)
	rgb(255,255,255)

Background Color

You can set the background color for HTML elements:

Hello World

Example

```
<h1 style="background-color:Blue;">Hello World</h1>  
<p style="background-color:Tomato;">Loremipsum...</p>
```

HTML Headings

HTML headings are defined with the `<h1>` to `<h6>` tags.

`<h1>` defines the most important heading. `<h6>` defines the least important heading:

Example

```
<h1>This is heading 1</h1>  
<h2>This is heading 2</h2>  
<h3>This is heading 3</h3>
```

Heading 1

Heading 2

Heading 3

Heading 4

Heading 5

Heading 6

HTML Paragraphs

HTML paragraphs are defined with the `<p>` tag:

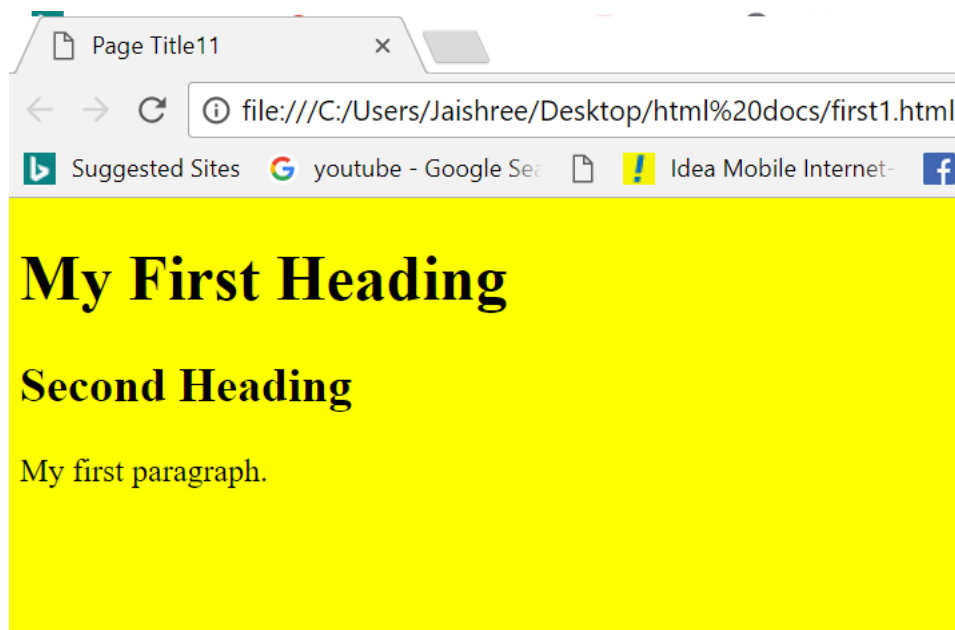
Example

```
<p>This is a paragraph.</p>  
<p>This is another paragraph.</p>
```

Example

```
<!DOCTYPE html>  
<html>  
<head>  
<title>Page Title11</title>  
</head>  
<body bgcolor = "yellow">  
<h1>My First Heading</h1>  
<h2>Second Heading</h2>  
<p>My first paragraph.</p>  
</body>  
</html>
```

Output:



HTML Text Formatting

HTML contains several elements for defining text with a special meaning.

Example

This text is bold

This text is italic

This is _{subscript} and ^{superscript}

HTML Formatting Elements

Formatting elements were designed to display special types of text:

- `<b>` - Bold text
- `<strong>` - Important text
- `<i>` - Italic text
- `<em>` - Emphasized text
- `<mark>` - Marked text
- `<small>` - Smaller text
- `<del>` - Deleted text
- `<ins>` - Inserted text
- `<sub>` - Subscript text
- `<sup>` - Superscript text

HTML `<b>` and `<strong>` Elements

The HTML `<b>` element defines bold text, without any extra importance.

Example

`<b>`This text is bold`</b>`

Meta tag: The `<meta>` tag defines metadata about an HTML document. Metadata is data (information) about data. `<meta>` tags always go inside the `<head>` element, and are typically used to specify character set, page description, keywords, author of the document, and viewport settings.

Metadata will not be displayed on the page, but is machine parsable. Metadata is used by browsers (how to display content or reload page), search engines (keywords), and other web services. There is a method to let web designers take control over the viewport (the user's visible area of a web page), through the `<meta>` tag (See "Setting The Viewport" example below).

Example: `<head>`

`<meta charset="UTF-8">`

`<meta name="description" content="kdu website">`

`<meta name="keywords" content="kebri dehar, university, education">`

```
<meta name="author" content="President">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
```

In meta tag the name attribute will tell what type of information it is and content is description of that type of information. Like The viewport is the user's visible area of a web page. It varies with the device - it will be smaller on a mobile phone than on a computer screen.

The `width=device-width` part sets the width of the page to follow the screen-width of the device (which will vary depending on the device). The `initial-scale=1.0` part sets the initial zoom level when the page is first loaded by the browser.

HTML Images

HTML images are defined with the `<img>` tag. The `<img>` tag is used to embed an image in an HTML page. Images are not technically inserted into a web page; images are linked to web pages. The `<img>` tag creates a holding space for the referenced image.

The `<img>` tag has two required attributes:

- `src` - Specifies the path to the image
- `alt` - Specifies an alternate text for the image, if the image for some reason cannot be displayed
- `width`, and `height` are provided as attributes:

Note: Also, always specify the width and height of an image. If width and height are not specified, the page might flicker while the image loads.

The src Attribute

The `<img>` tag is used to embed an image in an HTML page. The `src` attribute specifies the path to the image to be displayed:

The HR tag The `<hr>` tag in HTML stands for horizontal rule and is used to insert a horizontal rule or a thematic break in an HTML page to divide or separate document sections. The `<hr>` tag is an empty tag, and it does not require an end tag.

Tag Attributes: The table given below describe the `<hr>` tag attributes. **These attributes are not supported in HTML5:**

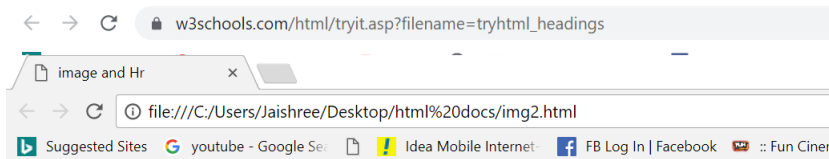
Attribute	Value	Description
<code>align</code>	left center right	Used to specify the alignment of the horizontal rule.
<code>noshade</code>	Noshade	Used to specify the bar without shading effect.
<code>size</code>	Pixels	Used to specify the height of the horizontal rule.

width Pixels Used to specify the width of the horizontal rule.

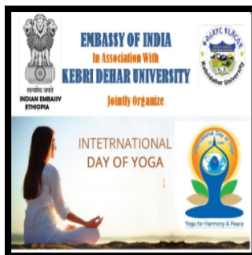
Example:

```
<!DOCTYPE html>
<html>
<head>
<title>image and Hr</title>
</head>
<body>
<h1>example of image</h1>
<img src = "POSTER.JPG" height = "200" width = "200" border=5><br><br>
<h2 style = "color : red">Given below is horizontal rule of size 1</h2>
<hr size="1">
<h2 style = "color : red">Given below is horizontal rule of size 10</h2>
<hr size="10">
<h2 style = "color : red">Given below is horizontal rule of size 1 and size 10 with noshade
</h2>
<hr size="10" noshade>
</body>
</html>
```

Output:



example of image



Given below is horizontal rule of size 1



Given below is horizontal rule of size 10



Given below is horizontal rule of size 1 and size 10 with noshade



HTML Links

HTML links are defined with the `<a>` tag:

The href Attribute

The `<a>` tag defines a hyperlink. The `href` attribute specifies the URL of the .page the link goes to:

Example

```
<a href="https://www.w3schools.com">Visit link W3Schools</a>
```

Image map: The HTML `<map>` tag defines an image map. An image map is an image with clickable areas. The areas are defined with one or more `<area>` tags. The idea behind an image map is that you should be able to perform different actions depending on where in the image you click. To create an image map you need an image, and some HTML code that describes the clickable areas.

Example:

```


<map name="workmap">
  <area shape="rect" coords="34,44,270,350" alt="Computer" href="computer.htm">
  <area shape="rect" coords="290,172,333,250" alt="Phone" href="phone.htm">
  <area shape="circle" coords="337,300,44" alt="Coffee" href="coffee.htm">
</map>
```

Note: in case rectangle it define coordinates of upper left and lower right corner. In case circle it define coordinates of center and length of radius

HTML Tables

The table in HTML consists of table cells inside rows and columns. A table is a structured set of data made up of rows and columns (**tabular data**). A table allows you to quickly and easily look up values that indicate some kind of connection between different types of data, for example a person and their age, or a day of the week, or the timetable for a local swimming pool.

The HTML tables are created using the `<table>` tag in which the

- ✓ `<tr>` tag is used to create table rows
- ✓ `<td>` tag is used to create data cells. The elements under `<td>` are regular and left aligned by default
- ✓ the border is an attribute of `<table>` tag and it is used to put a border across all the cells. If you do not need a border, then you can use `border = "0"`.
- ✓ Table heading can be defined using `<th>` tag. This tag will be put to replace `<td>` tag, which is used to represent actual data cell. Normally you will put your top row as table heading as shown below, otherwise you can use `<th>` element in any row. Headings, which are defined in `<th>` tag are centered and bold by default.

Example:

```
<!DOCTYPE html>
<html>
<body>
```

```
<h2>A basic HTML table</h2>
```

```
<table border = "5">
  <tr>
    <th>Company</th>
    <th>Contact</th>
    <th>Country</th>
  </tr>
  <tr>
    <td>Alfreds Futterkiste</td>
    <td>Maria Anders</td>
    <td>Germany</td>
  </tr>
  <tr>
    <td>Centro comercial Moctezuma</td>
    <td>Francisco Chang</td>
    <td>Mexico</td>
  </tr>
</table>
<p>To understand the example better, we have added borders to the table.</p>
</body>
</html>
```

Output

A basic HTML table

Company	Contact	Country
Alfreds Futterkiste	Maria Anders	Germany
Centro comercial Moctezuma	Francisco Chang	Mexico

To understand the example better, we have added borders to the table.

Cellpadding and Cellspacing Attributes

There are two attributes called *cellpadding* and *cellspacing* which you will use to adjust the white space in your table cells. The *cellspacing* attribute defines space between table cells, while *cellpadding* represents the distance between cell borders and the content within a cell.

Colspan and Rowspan Attributes

You will use **colspan** attribute if you want to merge two or more columns into a single column. Similar way you will use **rowspan** if you want to merge two or more rows.

Example

```

!DOCTYPE html>
<html>

<head>
<title>HTML Table Colspan/Rowspan</title>
</head>

<body>
<table border = "1">
<tr>
<th>Column 1</th>
<th>Column 2</th>
<th>Column 3</th>
</tr>
<tr>
<td rowspan = "2">Row 1 Cell 1</td>
<td>Row 1 Cell 2</td>
<td>Row 1 Cell 3</td>
</tr>
<tr>
<td>Row 2 Cell 2</td>
<td>Row 2 Cell 3</td>
</tr>
<tr>
<td colspan = "3">Row 3 Cell 1</td>
</tr>
</table>
</body>

</html>

```

Column 1	Column 2	Column 3
Row 1 Cell 1	Row 1 Cell 2	Row 1 Cell 3
	Row 2 Cell 2	Row 2 Cell 3
Row 3 Cell 1		

Table Height and Width

You can set a table width and height using **width** and **height** attributes. You can specify table width or height in terms of pixels or in terms of percentage of available screen area

```

<!DOCTYPE html>
<html>

<head>
<title>HTML Table Width/Height</title>
</head>

<body>
<table border = "1" width = "400" height = "150">
<tr>
<td>Row 1, Column 1</td>
<td>Row 1, Column 2</td>
</tr>

<tr>
<td>Row 2, Column 1</td>
<td>Row 2, Column 2</td>
</tr>
</table>
</body>

</html>

```

Table Header, Body, and Footer

Tables can be divided into three portions – a header, a body, and a foot. The head and foot are rather similar to headers and footers in a word-processed document that remain the same for every page, while the body is the main content holder of the table.

The three elements for separating the head, body, and foot of a table are –

- **<thead>** – to create a separate table header.
- **<tbody>** – to indicate the main body of the table.
- **<tfoot>** – to create a separate table footer.

A table may contain several **<tbody>** elements to indicate *different pages* or groups of data. But it is notable that **<thead>** and **<tfoot>** tags should appear before **<tbody>**

HTML Forms

An HTML form is used to collect user input. The user input is most often sent to a server for processing.

HTML Forms are required, when you want to collect some data from the site visitor. For example, during user registration you would like to collect information such as name, email address, credit card, etc.

A form will take input from the site visitor and then will post it to a back-end application such as CGI, ASP Script or PHP script etc. The back-end application will perform required processing on the passed data based on defined business logic inside the application.

It is a container type of element which can contain various form elements. There are various form elements available like text fields, textarea fields, drop-down menus, radio buttons, checkboxes, etc.

The HTML **<form>** tag is used to create an HTML form and it has following syntax

```
<form action = "Script URL" method = "GET|POST">  
  form elements like input, textarea etc.  
</form>
```

Example of form

First name:

Last name:

The <form> Element

The HTML `<form>` element is used to create an HTML form for user input:

```
<form>
.
form elements
.
</form>
```

The `<form>` element is a container for different types of input elements, such as: text fields, checkboxes, radio buttons, submit buttons, etc.

All the different form elements are covered in this chapter: [HTML Form Elements](#).

The <input> Element in form

The HTML `<input>` element is the most used form element.

An `<input>` element can be displayed in many ways, depending on the `type` attribute. different types of form controls that you can use to collect data using HTML form –

- Text Input Controls
- Checkboxes Controls
- Radio Box Controls
- Select Box Controls
- Clickable Buttons
- Submit and Reset Button

Here are some examples: Text Input Controls There are three types of text input used on forms –

- **Single-line text input controls** – This control is used for items that require only one line of user input, such as search boxes or names. They are created using HTML `<input>` tag.
- **Password input controls** – This is also a single-line text input but it masks the character as soon as a user enters it. They are also created using HTML `<input>` tag.
- **Multi-line text input controls** – This is used when the user is required to give details that may be longer than a single sentence. Multi-line input controls are created using HTML `<textarea>` tag.

Checkbox	Radio
Normal ☒	☐ Normal
Checked ☒	☒ Checked
Disabled ☐	☐ Disabled
Disabled & Checked ☒	☒ Checked & Disabled

Type	Description
<code><input type="text"></code>	Displays a single-line text input field
<code><input type="radio"></code>	Displays a radio button (for selecting one of many choices)
<code><input type="checkbox"></code>	Displays a checkbox (for selecting zero or more of many choices)
<code><input type="submit"></code>	Displays a submit button (for submitting the form)
<code><input type="button"></code>	Displays a clickable button

The <label> Element:

The `<label>` tag defines a label for many form elements.

The `<label>` element is useful for screen-reader users, because the screen-reader will read out loud the label when the user focus on the input element.

The `<label>` element also help users who have difficulty clicking on very small regions (such as radio buttons or checkboxes) - because when the user clicks the text within the `<label>` element, it toggles the radio button/checkbox.

The `for` attribute of the `<label>` tag should be equal to the `id` attribute of the `<input>` element to bind them together.

Text Fields

The `<input type="text">` defines a single-line input field for text input.

Radio Buttons

The `<input type="radio">` defines a radio button.

Radio buttons let a user select ONE of a limited number of choices.

Checkboxes

The `<input type="checkbox">` defines a **checkbox**.

Checkboxes let a user select ZERO or MORE options of a limited number of choices.

The Submit Button

The `<input type="submit">` defines a button for submitting the form data to a form-handler.

The form-handler is typically a file on the server with a script for processing input data. The form-handler is specified in the form's **action** attribute.

The <select> Element

The **<select>** element defines a drop-down list:

Example of form, label and textbox

```
<!DOCTYPE html>
<html>
<body>
<h2>HTML Forms</h2>
<form action="/action_page.php">
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" value="John"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" value="Doe"><br><br>
  <input type="submit" value="Submit">
</form>
<p>If you click the "Submit" button, the form-data will be sent to a page called
"/action_page.php".</p>
</body>
</html>
```

Output:

HTML Forms

First name:

Last name:

If you click the "Submit" button, the form-data will be sent to a page called "/action_page.php".

Example for select element

```
<!DOCTYPE html>
<html>
<body>

<h1>The select element</h1>
```

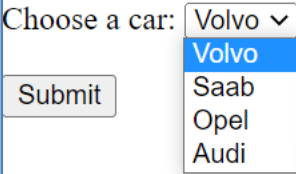
<p>The select element is used to create a drop-down list.</p>

```
<form action="/action_page.php">
  <label for="cars">Choose a car:</label>
  <select name="cars" id="cars">
    <option value="volvo">Volvo</option>
    <option value="saab">Saab</option>
    <option value="opel">Opel</option>
    <option value="audi">Audi</option>
  </select>
  <br><br>
  <input type="submit" value="Submit">
</form>
<br><br>
<p>Click the "Submit" button and the form-data will be sent to a page on the
server called "action_page.php".</p>
</body>
</html>
```

Output

The select element

The select element is used to create a drop-down list.

Choose a car: 

Click the "Submit" button and the form-data will be sent to a page on the server called "action_page.php".

Example for radio button

```
<!DOCTYPE html>
<html>
<body>
<h2>Radio Buttons</h2>
<p>Choose your favorite Web language:</p>
<form>
  <input type="radio" id="html" name="fav_language" value="HTML">
  <label for="html">HTML</label><br>
  <input type="radio" id="css" name="fav_language" value="CSS">
  <label for="css">CSS</label><br>
```

```
<input type="radio" id="javascript" name="fav_language" value="JavaScript">
<label for="javascript">JavaScript</label>
</form>
</body>
</html>
```

Radio Buttons

Choose your favorite Web language:

- ☐ HTML
- ☐ CSS
- ☐ JavaScript

Example for check box

```
<!DOCTYPE html>
<html>
<body>
<h2>Checkboxes</h2>
<p>The <strong>input type="checkbox"</strong> defines a checkbox:</p>
<form action="/action_page.php">
  <input type="checkbox" id="vehicle1" name="vehicle1" value="Bike">
  <label for="vehicle1"> I have a bike</label><br>
  <input type="checkbox" id="vehicle2" name="vehicle2" value="Car">
  <label for="vehicle2"> I have a car</label><br>
  <input type="checkbox" id="vehicle3" name="vehicle3" value="Boat">
  <label for="vehicle3"> I have a boat</label><br><br>
  <input type="submit" value="Submit">
</form>
</body>
</html>
```

Checkboxes

The **input type="checkbox"** defines a checkbox:

- ☐ I have a bike
- ☐ I have a car
- ☐ I have a boat

HTML Styles - CSS

CSS is the acronym for "**Cascading Style Sheet**".

CSS is used to control the style of a web document in a simple and easy way.

It can control the layout of multiple web pages all at once, hence CSS saves a lot of work.

CSS Solved a Big Problem

HTML was NEVER intended to contain tags for formatting a web page, HTML was created to describe the content of a web page, like:

```
<h1>This is a heading</h1>
```

```
<p>This is a paragraph.</p>
```

When tags like , and color attributes were added to the HTML 3.2 specification, it started a nightmare for web developers. Development of large websites, where fonts and color information were added to every single page, became a long and expensive process.

To solve this problem, the World Wide Web Consortium (W3C) created CSS.

CSS removed the style formatting from the HTML page!

With CSS, you can control the color, font, the size of text, the spacing between elements, how elements are positioned and laid out, what background images or background colors are to be used, different displays for different devices and screen sizes, and much more!

The word **cascading** means that a style applied to a parent element will also apply to all children elements within the parent. So, if you set the color of the body text to "blue", all headings, paragraphs, and other text elements within the body will also get the same color (unless you specify something else)!

Why to Learn CSS?

Cascading Style Sheets, fondly referred to as **CSS**, is a simple design language intended to simplify the process of making web pages presentable.

CSS is a MUST for students and working professionals to become a great Software Engineer specially when they are working in Web Development Domain. I will list down some of the key advantages of learning CSS:

- **CSS Saves a Lot of Work!** :The style definitions are normally saved in external .css files, With an external stylesheet file, you can change the look of an entire website by changing just one file!
- **Create Stunning Web site** - CSS handles the look and feel part of a web page. Using CSS, you can control the color of the text, the style of fonts, the spacing between paragraphs, how columns are sized and laid out, what background images or colors are used, layout designs, variations in display for different devices and screen sizes as well as a variety of other effects.
- **Become a web designer** - If you want to start a career as a professional web designer, HTML and CSS designing is a must skill.
- **Control web** - CSS is easy to learn and understand but it provides powerful control over the presentation of an HTML document. Most commonly, CSS is combined with the markup languages HTML or XHTML.
- **Learn other languages** - Once you understand the basic of HTML and CSS then other related technologies like javascript, php, or angular are become easier to understand.

Applications of CSS

As mentioned before, CSS is one of the most widely used style language over the web. I'm going to list few of them here:

- **CSS saves time** - You can write CSS once and then reuse same sheet in multiple HTML pages. You can define a style for each HTML element and apply it to as many Web pages as you want.
- **Pages load faster** - If you are using CSS, you do not need to write HTML tag attributes every time. Just write one CSS rule of a tag and apply it to all the occurrences of that tag. So less code means faster download times.
- **Easy maintenance** - To make a global change, simply change the style, and all elements in all the web pages will be updated automatically.
- **Superior styles to HTML** - CSS has a much wider array of attributes than HTML, so you can give a far better look to your HTML page in comparison to HTML attributes.
- **Multiple Device Compatibility** - Style sheets allow content to be optimized for more than one type of device. By using the same HTML document, different versions of a website can be presented for handheld devices such as PDAs and cell phones or for printing.
- **Global web standards** - Now HTML attributes are being deprecated and it is being recommended to use CSS. So it's a good idea to start using CSS in all the HTML pages to make them compatible to future browsers

Example: Simple example to change style of Hello world by CSS

```
<!DOCTYPE html>
<html>
  <head>
    <title>This is document title</title>
    <style>
      h1 {
        color: #36CFFF;
      }
    </style>
```

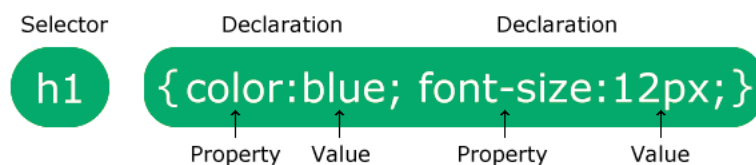
```
</head>
<body>
  <h1>Hello World!</h1>
</body>
</html>
```

Note: the css code is written in header section in above example. The code will change the colour of h1 heading in html document.

CSS Syntax

CSS rule consists of a selector and a declaration block.

Syntax



- ✓ The selector points to the HTML element you want to style.
- ✓ CSS block/(declaration block) is the code written between { }
- ✓ The declaration block contains one or more declarations separated by semicolons.
- ✓ Each declaration includes a CSS property name and a value, separated by a colon.
- ✓ Multiple CSS declarations are separated with semicolons, and declaration blocks are surrounded by curly braces.

Using CSS

CSS can be added to HTML documents in 3 ways:

- **Inline** - by using the **style** attribute inside HTML elements
- **Internal** - by using a **<style>** element in the **<head>** section
- **External** - by using a **<link>** element to link to an external CSS file

The most common way to add CSS, is to keep the styles in external CSS files. However, in this chapter we will use inline and internal styles, because this is easier to demonstrate, and easier for you to try it yourself.

Inline CSS

An inline CSS is used to apply a unique style to a single HTML element. An inline CSS uses the **style** attribute of an HTML element. The following example sets the text color of the **<h1>** element to blue, and the text color of the **<p>** element to red:

Example:

```
<!DOCTYPE html>
<html>
<body>
<h1 style="color:blue;text-align:center;">This is a heading</h1>
<p style="color:red;">This is a paragraph.</p>
</body>
</html>
```

Internal CSS

An internal CSS is used to define a style for a single HTML page. An internal CSS is defined in the **<head>** section of an HTML page, within a **<style>** element. The following example sets the text color of ALL the **<h1>** elements (on that page) to blue, and the text color of ALL the **<p>** elements to red. In addition, the page will be displayed with a "powderblue" background color:

Example

```
<!DOCTYPE html>
<html>
<head>
<style>
body {background-color: powderblue;}
h1 {color: blue;}
p {color: red;}
</style>
</head>
<body>
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
</body>
</html>
```

External CSS An external style sheet is used to define the style for many HTML pages. To use an external style sheet, add a link to it in the **<head>** section of each HTML page using the link element:

Link Element : External style sheets can be referenced with a full URL or with a path relative to the current web page.

Example: This example uses a full URL to link to a style sheet:

```
<link rel="stylesheet" href="https://www.w3schools.com/html/styles.css">
```

Example 2: This example links to a style sheet located in the html folder on the current web site:

```
<link rel="stylesheet" href="/html/styles.css">
```

The rel Attribute

rel is the relationship between the external file and the current file. For CSS, you use stylesheet. For example, rel="stylesheet".

The type Attribute

type is the type of the document you are linking to the HTML. For CSS, it is text/css. For example type="text/css".

The href Attribute

href stands for "hypertext reference". You use it to specify the location of the CSS file and the file name. It is a clickable link, so you can also hold CTRL and click it to view the CSS file.

For example, href="styles.css" if the CSS file is located in the same folder as the HTML file. Or href="folder/styles.css" if the CSS file is located on another folder.

Example

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
</body>
</html>
```

The external style sheet can be written in any text editor. The file must not contain any HTML code, and must be saved with a .css extension. Here is what the "styles.css" file looks like:

"styles.css":

```
body {
  background-color: powderblue;
}
h1 {
  color: blue;
}
p {
  color: red;
}
```

With an external style sheet, you can change the look of an entire web site, by changing one file!

CSS Selectors

A CSS selector selects the HTML element(s) you want to style. CSS selectors are used to "find" (or select) the HTML elements you want to style. We can divide CSS selectors into five categories:

1. Simple selectors (select elements based on name, id, class)
2. [Combinator selectors](#) (select elements based on a specific relationship between them)
3. [Pseudo-class selectors](#) (select elements based on a certain state)
4. [Pseudo-elements selectors](#) (select and style a part of an element)
5. [Attribute selectors](#) (select elements based on an attribute or attribute value)

1. Simple selectors:

Element Selector: The element selector selects HTML elements based on the element name. Here, all <p> elements on the page will be center-aligned, with a red text color:

```
p {  
  text-align: center;  
  color: red;  
}
```

The CSS id Selector: The id selector uses the id attribute of an HTML element to select a specific element. The id of an element is unique within a page, so the id selector is used to select one unique element! To select an element with a specific id, write a hash (#) character, followed by the id of the element.

```
#para1 {  
  text-align: center;  
  color: red;  
}
```

The CSS class Selector : The class selector selects HTML elements with a specific class attribute. To select elements with a specific class, write a period (.) character, followed by the class name.

```
.center {  
  text-align: center;  
  color: red;  
}
```

You can also specify that only specific HTML elements should be affected by a class.

```
p.center {  
  text-align: center;  
  color: red;  
}
```

HTML elements can also refer to more than one class. In this example the <p> element will be styled according to class="center" and to class="large":

```
<p class="center large">This paragraph refers to two classes.</p>
```

Note: A class name cannot start with a number!

The CSS Grouping Selector

The grouping selector selects all the HTML elements with the same style definitions. Look at the following CSS code (the h1, h2, and p elements have the same style definitions):

```
h1 {  
    text-align: center;  
    color: red;  
}
```

```
h2 {  
    text-align: center;  
    color: red;  
}
```

```
p {  
    text-align: center;  
    color: red;  
}
```

- It will be better to group the selectors, to minimize the code.
- To group selectors, separate each selector with a comma.

In this example we have grouped the selectors from the code above:

```
h1, h2, p {  
    text-align: center;  
    color: red;  
}
```

The CSS Universal Selector: The universal selector (*) selects all HTML elements on the page. The CSS rule below will affect every HTML element on the page:

```
* {  
    text-align: center;  
    color: blue;  
}
```

Full example: The below example will use the selector in html header section, all selectors can be use like this or even can be used in external file.

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
* {  
    text-align: center;
```

```
    color: blue;
}
</style>
</head>
<body>
<h1>Hello world!</h1>
<p>Every element on the page will be affected by the style.</p>
<p id="para1">Me too!</p>
<p>And me!</p>
</body>
</html>
```

Properties that can be changed by the CSS:

1. CSS Colors, Fonts and Sizes

- Here, we will demonstrate some commonly used CSS properties.
- The CSS **color** property defines the text color to be used.
- The CSS **font-family** property defines the font to be used.
- The CSS **font-size** property defines the text size to be used.

Example

Use of CSS color, font-family and font-size properties:

```
<!DOCTYPE html>
<html>
<head>
<style>
h1 {
    color: blue;
    font-family: verdana;
    font-size: 300%;
}
p {
    color: red;
    font-family: courier;
    font-size: 160%;
}
</style>
</head>
<body>
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
</body>
</html>
```

2. CSS Border:

The CSS **border** property defines a border around an HTML element.

Tip: You can define a border for nearly all HTML elements.

Example Use of CSS border property:

```
p {
  border: 2px solid powderblue;
}
```

3. CSS Padding

The CSS **padding** property defines a padding (space) between the text and the border.

Example : Use of CSS border and padding properties:

```
p {
  border: 2px solid powderblue;
  padding: 30px;
}
```

4. CSS Margin

The CSS **margin** property defines a margin (space) outside the border.

Example

Use of CSS border and margin properties:

```
p {
  border: 2px solid powderblue;
  margin: 50px;
}
```

Apart from above properties the CSS can change several other properties, below is short description given in short

Text Properties

Property	Description	Values
color	Sets the color of a text	RGB, hex, keyword
line-height	Sets the distance between lines	normal, <i>number</i> , <i>length</i> , %
letter-spacing	Increase or decrease the space between characters	normal, <i>length</i>
text-align	Aligns the text in an element	left, right, center, justify
text-decoration	Adds decoration to text	none, underline, overline, line-through
text-indent	Indents the first line of text in an element	<i>length</i> , %
text-transform	Controls the letters in an element	none, capitalize, uppercase, lowercase

Font Properties

Property	Description	Values
font	Sets all the font properties in one declaration	<i>font-style</i> , <i>font-variant</i> , <i>font-weight</i> , <i>font-size/line-height</i> , <i>font-family</i> , caption, icon, menu, message-box, small-caption, status-bar, inherit
font-family	Specifies the font family for text	<i>family-name</i> , <i>generic-family</i> , inherit
font-size	Specifies the font size of text	xx-small, x-small, small, medium, large, x-large, xx-large, smaller, larger, <i>length</i> , %, inherit
font-style	Specifies the font style for text	normal, italic, oblique, inherit
font-variant	Specifies whether or not a text should be displayed in a small-caps font	normal, small-caps, inherit
font-weight	Specifies the weight of a font	normal, bold, bolder, lighter, 100, 200, 300, 400, 500, 600, 700, 800, 900, inherit Careful, many of these are not supported!

List Properties

Property	Description	Values
list-style	Sets all the properties for a list in one declaration	<i>list-style-type</i> , <i>list-style-position</i> , <i>list-style-image</i> , inherit
list-style-image	Specifies an image as the list-item marker	URL, none, inherit
list-style-position	Specifies where to place the list-item marker	inside, outside, inherit
list-style-type	Specifies the type of list-item marker	none, disc, circle, square, decimal, decimal-leading-zero, armenian, georgian, lower-alpha, upper-alpha, lower-greek, lower-latin, upper-latin, lower-roman, upper-roman, inherit

Border properties:

Property	Description	Values
border	Sets all the border properties in one declaration	<i>border-width</i> , <i>border-style</i> , <i>border-color</i>
border-bottom	Sets all the bottom border properties in one declaration	<i>border-bottom-width</i> , <i>border-bottom-style</i> , <i>border-bottom-color</i>
border-bottom-color	Sets the color of the bottom border	<i>border-color</i>
border-bottom-style	Sets the style of the bottom border	<i>border-style</i>
border-bottom-width	Sets the width of the bottom border	<i>border-width</i>
border-color	Sets the color of the four borders	<i>color_name</i> , <i>hex_number</i> , <i>rgb_number</i> , transparent, inherit
border-left	Sets all the left border properties in one declaration	<i>border-left-width</i> , <i>border-left-style</i> , <i>border-left-color</i>
border-left-color	Sets the color of the left border	<i>border-color</i>
border-left-style	Sets the style of the left border	<i>border-style</i>
border-left-width	Sets the width of the left border	<i>border-width</i>
border-right	Sets all the right border properties in one declaration	<i>border-right-width</i> , <i>border-right-style</i> , <i>border-right-color</i>
border-right-color	Sets the color of the right border	<i>border-color</i>
border-right-style	Sets the style of the right border	<i>border-style</i>
border-right-width	Sets the width of the right border	<i>border-width</i>
border-style	Sets the style of the four borders	none, hidden, dotted, dashed, solid, double, groove, ridge, inset, outset, inherit
border-top	Sets all the top border properties in one declaration	<i>border-top-width</i> , <i>border-top-style</i> , <i>border-top-color</i>
border-top-color	Sets the color of the top border	<i>border-color</i>
border-top-style	Sets the style of the top border	<i>border-style</i>
border-top-width	Sets the width of the top border	<i>border-width</i>
border-width	Sets the width of the four borders	thin, medium, thick, <i>length</i> , inherit

Note: Like above, all elements have different properties that can be changed using CSS.

CSS Color

Names Supported by All Browsers All modern browsers support the following 140 color names.

CSS Colors values: **Colors in CSS can be specified by the following methods:**

- Hexadecimal colors
- Predefined/Cross-browser color names
- RGB colors

The main color techniques are given above and remaining are extension of these techniques like.

- Hexadecimal colors with transparency
- RGBA colors
- HSL colors
- HSLA colors
- With the **currentColor** keyword

Hexadecimal Colors

A hexadecimal color is specified with: #RRGGBB, where the RR (red), GG (green) and BB (blue) hexadecimal integers specify the components of the color. All values must be between 00 and FF.

For example, the #0000ff value is rendered as blue, because the blue component is set to its highest value (ff) and the others are set to 00.

RGB Colors

An RGB color value is specified with the [rgb\(\) function](#), which has the following syntax:

CSS RGB Colors (red, green, blue)

Each parameter (red, green, and blue) defines the intensity of the color and can be an integer between 0 and 255 or a percentage value (from 0% to 100%).

For example, the rgb(0,0,255) value is rendered as blue, because the blue parameter is set to its highest value (255) and the others are set to 0.

Also, the following values define equal color: rgb(0,0,255) and rgb(0%,0%,100%).

Predefined/Cross-browser Color Names

140 color names are predefined in the HTML and CSS color specification.

For example: **blue**, **red**, **coral**, **brown**, etc:

Media-dependent cascades:

One of the most important features of style sheets is that they specify how a document is to be presented on different media: on the screen, on paper, with a speech synthesizer, with a braille device, etc.

Certain CSS properties are only designed for certain media (e.g., the '[page-break-before](#)' property only applies to paged media). On occasion, however, style sheets for different media types may share a property, but require different values for that property.

For example, the '[font-size](#)' property is useful both for screen and print media. The two media types are different enough to require different values for the common property; a document will typically need a larger font on a computer screen than on paper. Therefore, it is necessary to express that a style sheet, or a section of a style sheet, applies to certain media types.

Recognized media types

The names chosen for CSS media types reflect target devices for which the relevant properties make sense. In the following list of CSS media types the names of media types are normative, but the descriptions are informative. Likewise, the "Media" field in the description of each property is informative.

all: Suitable for all devices.

braille: Intended for braille tactile feedback devices.

embossed: Intended for paged braille printers.

handheld: Intended for handheld devices (typically small screen, limited bandwidth).

print: Intended for paged material and for documents viewed on screen in print preview mode. Please consult the section on [paged media](#) for information about formatting issues that are specific to paged media.

projection: Intended for projected presentations, for example projectors. Please consult the section on [paged media](#) for information about formatting issues that are specific to paged media.

screen

Intended primarily for color computer screens.

speech

Intended for speech synthesizers. Note: CSS2 had a similar media type called 'aural' for this purpose. See the appendix on [aural style sheets](#) for details.

tty

Intended for media using a fixed-pitch character grid (such as teletypes, terminals, or portable devices with limited display capabilities). Authors should not use [pixel units](#) with the "tty" media type.

tv

Intended for television-type devices (low resolution, color, limited-scrollability screens, sound available).

JavaScript

Scripting language: Both scripting and programming are computer languages, with all the scripting languages being programming languages, but all programming languages are not scripting languages. *The basic difference between a scripting language and a programming language is that scripting languages do not need an additional step of compilation and rather they are interpreted, whereas programming languages are compiled and hence need a compilation step to convert the high-level language to machine code.*

"A scripting language is a type of programming language which does not require explicit compilation step, and it is designed for a runtime system to automate the execution of tasks." For example, a JavaScript program is not needed to be compiled before we run it. These are also known as very high-level programming languages because of working at a high level of abstraction.

Scripting languages support "**script**," which is small program written for a specific runtime environment. These are interpreted at runtime rather than compiled. It means, to convert the source code to machine code, scripting languages use an interpreter, not the compiler. As scripting language is not compiled so as we write something meaningful, we can run it immediately.

The scripting language refers to dynamic high-level, general-purpose interpreted languages such as Python, Perl, etc. Thus, a scripting language can automate different environments such as **application softwares, webpages, text editors, operating system shells, computer games, etc.**

Advantages

- It is an easy and quick process to learn coding in Scripting language, and for this, much knowledge of web technology is not needed.
- In scripting languages, a wide variety of libraries is available that enable the developers to develop new applications.
- With the help of scripting languages, we can add visualization interfaces and combinations to web pages. Most of the latest web pages need scripting languages for creating enhanced web pages, fascinating UI, and many more.
- There are less number of data structures and variable to be used, which make it highly efficient.
- These are Less code-intensive as compared to traditional programming languages.

Differences between programming and scripting language:

Parameters	Scripting Language	Programming Language
Language Type	The scripting languages are interpreter-based languages.	The programming languages are compiler-based languages.
Use	The scripting languages help in combining the existing components of an application.	The programming languages help in developing anything from scratch.
Running of Language	A user needs to run scripting languages inside an existing program. Thus, it's program-dependent.	Programming languages are program-independent.
Conversion	Scripting languages convert high-level instructions into machine language.	Programming languages help in converting the full program into the machine language (at once).
Compilation	You don't need to compile these languages.	These languages first need a compilation.
Design	These make the coding process simple and fast.	These provide full usage of the languages.
File Type	Scripting languages don't create any file types.	Programming languages create .exe files.
Complexity	These are very easy to use and easy to write.	These are pretty complex in terms of writing and usage.

Type of Coding	Scripting languages help write a small piece of an entire code.	Programming languages help write the full code concerning a program.
Developing Time	These take less time because they involve lesser code.	These take more time because a programmer must write the entire code.
Interpretation	We usually interpret a scripting language in another program.	The compile results of a programming language are stand-alone. No other program needs to interpret it.
Requirement of Host	Scripting languages require hosts for execution.	Programming languages are self-executable. They don't require any host.
Length of Codes	These involve very few and short coding lines.	These require numerous lines of coding for a single function.
Support	These provide limited support to data types, user interface design, and graphic design.	These provide rich support for graphic design, data types, and user interface design.
Maintenance	These involve very low maintenance.	These involve high maintenance.
Cost	It is easier and cheaper to maintain a scripting language.	Maintaining a programming language is comparatively more expensive.
Example	VB Script, Perl, Ruby, PHP, JavaScript, etc.	C, C++, COBOL, Basic, VB, C#, Pascal, Java, etc.

JavaScript is a lightweight, interpreted programming language. It is designed for creating network-centric applications. It is complimentary to and integrated with Java. JavaScript is very easy to implement because it is integrated with HTML. It is open and cross-platform.

Why to Learn Javascript?

Mostly we use javascript for client side programming.

- JavaScript allows you to add interactivity to a web page. Typically, you use JavaScript with HTML and CSS to enhance a web page's functionality, such as [validating forms](#), creating interactive maps, and displaying animated charts.
- When a web page is loaded, i.e., after HTML and CSS have been downloaded, the JavaScript engine in the web browser executes the JavaScript code. The JavaScript code then modifies the HTML and CSS to update the user interface dynamically.

Javascript is a MUST for students and working professionals to become a great Software Engineer specially when they are working in Web Development Domain. I will list down some of the key advantages of learning Javascript:

- Javascript is the most popular programming language in the world and that makes it a programmer's great choice. Once you learnt Javascript, it helps you developing great front-end as well as back-end softwares using different Javascript based frameworks like jQuery, Node.JS etc.
- Javascript is everywhere, it comes installed on every modern web browser and so to learn Javascript you really do not need any special environment setup. For example Chrome, Mozilla Firefox, Safari and every browser you know as of today, supports Javascript.
- Javascript helps you create really beautiful and crazy fast websites. You can develop your website with a console like look and feel and give your users the best Graphical User Experience.
- JavaScript usage has now extended to mobile app development, desktop app development, and game development. This opens many opportunities for you as Javascript Programmer.
- Due to high demand, there is tons of job growth and high pay for those who know JavaScript. You can navigate over to different job sites to see what having JavaScript skills looks like in the job market.

- Great thing about Javascript is that you will find tons of frameworks and Libraries already developed which can be used directly in your software development to reduce your time to market.

Applications of Javascript Programming

As mentioned before, **Javascript** is one of the most widely used **programming languages** (Front-end as well as Back-end). It has its presence in almost every area of software development. I'm going to list few of them here:

- **Client side validation** - This is really important to verify any user input before submitting it to the server and Javascript plays an important role in validating those inputs at front-end itself.
- **Manipulating HTML Pages** - Javascript helps in manipulating HTML page on the fly. This helps in adding and deleting any HTML tag very easily using javascript and modify your HTML to change its look and feel based on different devices and requirements.
- **User Notifications** - You can use Javascript to raise dynamic pop-ups on the webpages to give different types of notifications to your website visitors.
- **Back-end Data Loading** - Javascript provides Ajax library which helps in loading back-end data while you are doing some other processing. This really gives an amazing experience to your website visitors.
- **Presentations** - JavaScript also provides the facility of creating presentations which gives website look and feel. JavaScript provides RevealJS and BespokeJS libraries to build a web-based slide presentations.
- **Server Applications** - Node JS is built on Chrome's Javascript runtime for building fast and scalable network applications. This is an event based library which helps in developing very sophisticated server applications including Web Servers.

Example of Hello world in java script

```
<html>
<body>
  <script language = "javascript" type = "text/javascript">

    document.write("Hello World!")
  </script>
</body>
</html>
```

Where and how to use JavaScript In HTML

The HTML **<script>** tag is used to define a client-side script (JavaScript). JavaScript code is inserted between **<script>** and **</script>** tags.

The **<script>** element either contains script statements, or it points to an external script file through the **src** attribute. Common uses for JavaScript are image manipulation, form validation, and dynamic changes of content.

Example

```
<script>
document.write("Hello World!")
</script>
```

Old JavaScript examples may use a type attribute: `<script type="text/javascript">`.

The type attribute is not required. JavaScript is the default scripting language in HTML.

Java script can be used in three ways

1. In header section
2. In Body of html document
3. As external javascript file

You can place any number of scripts in an HTML document. Scripts can be placed in the `<body>`, or in the `<head>` section of an HTML page, or in both.

1. JavaScript in `<head>`

In this example, a JavaScript **function** is placed in the `<head>` section of an HTML page. The function is invoked (called) when a button is clicked:

```
<!DOCTYPE html>
<html>
<head>
<script>
function myFunction() {
  document.write ("hello through header");
}
</script>
</head>

<body>
<h2>Demo JavaScript in Head</h2>
<p id="demo">A Paragraph</p>
<button type="button" onclick="myFunction()">Try it</button>
</body>
</html>
```

2. JavaScript in `<body>`

In this example, a JavaScript **function** is placed in the `<body>` section of an HTML page. The function is invoked (called) when a button is clicked:

Example

```
<!DOCTYPE html>
<html>
<body>
<h2>Demo JavaScript in Body</h2>
<p id="demo">A Paragraph</p>
<button type="button" onclick="myFunction()">Try it</button>
<script>
function myFunction() {
  document.write ("hello through body");
}
</script>
</body>
</html>
```

A JavaScript **function** is a block of JavaScript code, that can be executed when "called" for. For example, a function can be called when an **event** occurs, like when the user clicks a button.

3. External JavaScript

- JavaScripts can also be placed in external files: External scripts are practical when the same code is used in many different web pages.
- JavaScript files have the file extension **.js**. To use an external script, put the name of the script file in the **src** (source) attribute of a **<script>** tag:
- You can place an external script reference in **<head>** or **<body>** as you like.
- The script will behave as if it was located exactly where the **<script>** tag is located.
- External scripts cannot contain **<script>** tags.

External JavaScript Advantages

Placing scripts in external files has some advantages:

- It separates HTML and code
- It makes HTML and JavaScript easier to read and maintain
- Cached JavaScript files can speed up page loads
- To add several script files to one page - use several script tags:

Example: HTML File

```
<!DOCTYPE html>
<html>
<body>
<h2>Demo External JavaScript</h2>
<p>A Paragraph.</p>
<button type="button" onclick="myFunction()">Try it</button>
<p>This example links to "myScript.js".</p>
<p>(myFunction is stored in "myScript.js")</p>
<script src="myScript.js"></script>
</body>
</html>
```

myScript.js

```
function myFunction()
{
  document.write("hello through external file");
}
```

JavaScript Display (output) Possibilities

JavaScript can "display" data in different ways:

- Writing into an HTML element, using **innerHTML**.
- Writing into the HTML output using **document.write()**.
- Writing into an alert box, using **window.alert()**.
- Writing into the browser console, using **console.log()**.

Example:

```
!DOCTYPE html>
<html>
<body>
<h1>My First Web Page</h1>
<p>My first paragraph.</p>
<button type="button" onclick="document.write(5 + 6)">Try it</button>
</body>
</html>
```

Javascript program and statement: **JavaScript program** is a list of programming **statements**.

In HTML, JavaScript programs are executed by the web browser.

JavaScript statements are composed of: Values, Operators, Expressions, Keywords, and Comments. Most JavaScript programs contain many JavaScript statements. The statements are executed, one by one, in the same order as they are written.

JavaScript programs (and JavaScript statements) are often called JavaScript code.

Semicolons ; Semicolons separate JavaScript statements. Add a semicolon at the end of each executable statement:

Examples

```
let a, b, c; // Declare 3 variables
a = 5;      // Assign the value 5 to a
b = 6;      // Assign the value 6 to b
c = a + b;  // Assign the sum of a and b to c
```

When separated by semicolons, multiple statements on one line are allowed:

```
a = 5; b = 6; c = a + b;
```

On the web, you might see examples without semicolons.

Ending statements with semicolon is not required, but highly recommended.

JavaScript Variables In a programming language, **variables** are used to **store** data values. JavaScript uses the keywords **var**, **let** to declare variable and **const** to **declare** constants. An **equal sign** is used to **assign values** to variables. In this example, x is defined as a variable. Then, x is assigned (given) the value 6:

```
let x;
x = 6;
```

/ How to create variables:

```
var x;
```

```
let y;
```

// How to use variables:

```
x = 5;
```

```
y = 6;
```

```
let z = x + y;
```

Numbers are written with or without decimals:

10.50

1001

Strings are text, written within double or single quotes:

"John Doe"

'John Doe'

Comments: Single line comments start with **//**. Any text between **//** and the end of the line will be ignored by JavaScript (will not be executed).

Multi-line comments start with **/*** and end with ***/**. Any text between **/*** and ***/** will be ignored by JavaScript.

Types of JavaScript Operators There are different types of JavaScript operators:

- Arithmetic Operators
- Increment and decrement operator
- Assignment Operators
- Comparison Operators
- Logical Operators
- Conditional Operators
- Type Operators

Exponent operator is denoted by ****** : $Z = X^{**}Y$ // $z = x^y$
 $X^{**} = Y$ will denote $X = X^{**}Y$

Javascript Array: An array is a special variable, which can hold more than one value: An array can hold many values under a single name, and you can access the values by referring to an index number. Using an array literal is the easiest way to create a JavaScript Array. Syntax:

```
const array_name = [item1, item2, ...];
```

It is a common practice to declare arrays with the **const** keyword.

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Arrays</h2>
<p id="demo"></p>
<script>
const cars = [];
cars[0]= "Saab";
cars[1]= "Volvo";
cars[2]= "BMW";
document.getElementById("demo").innerHTML = cars;
</script>
</body>
</html>
```

Array by new keyword:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Arrays</h2>
<p id="demo"></p>
<script>
const cars = new Array("Saab", "Volvo", "BMW");
document.getElementById("demo").innerHTML = cars;
</script>
</body>
</html>
```

You access an array element by referring to the **index number**:

```
const cars = ["Saab", "Volvo", "BMW"];
let car = cars[0];
```

Conditional and loop statement in javascript

- **Conditional:** Just like C/C++/Java javascript support if, if-else, if-else-if and switch statement.
- **Iterative:** Java script have same syntax for do-while, while and for loop statement.
- **Break and continue:** javascript also support break and continue statements.

Javascript objects

Arrays are a special type of objects. The **typeof** operator in JavaScript returns "object" for arrays. But, JavaScript arrays are best described as arrays. Arrays use **numbers** to access its "elements". In this example, **person[0]** returns John:

Array:

```
const person = ["John", "Doe", 46];
```

Objects use **names** to access its "members". In this example, `person.firstName` returns John in following example:

```
const person = {firstName:"John", lastName:"Doe", age:46};
```

Full example:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Objects</h2>
<p>JavaScript uses names to access object properties.</p>
<p id="demo"></p>
<script>
const person = {firstName:"John", lastName:"Doe", age:46};
document.getElementById("demo").innerHTML = person.firstName;
</script>
</body>
</html>
```

Javascript functions: JavaScript functions are **defined** with the **function** keyword. You can use a function **declaration** or a function **expression**.

```
function functionName(parameters) {
  // code to be executed
}
```

Declared functions are not executed immediately. They are "saved for later use", and will be executed later, when they are invoked (called upon).

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Functions</h2>
<p>This example calls a function which performs a calculation and returns the result:</p>
<p id="demo"></p>
<script>
var x = myFunction(4, 3);
document.getElementById("demo").innerHTML = x;
function myFunction(a, b) {
  return a * b;
}
</script>
</body>
</html>
```

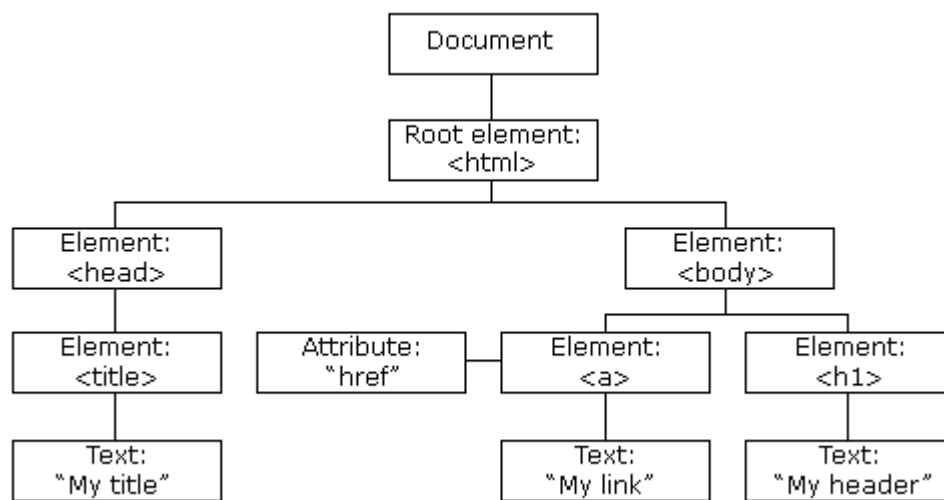
JavaScript- HTML DOM Model

The HTML DOM (Document Object Model): When a web page is loaded, the browser will create object for every element of html and creates **Document Object Model** of the page. The **HTML DOM** model is constructed as a tree of Objects:

DOM: The DOM is a W3C (World Wide Web Consortium) standard. The DOM defines a standard for accessing documents: *"The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document."*

- HTML DOM methods are **actions** you can perform (on HTML Elements).
- HTML DOM properties are **values** (of HTML Elements) that you can set or change.
- **The DOM Programming Interface**
- The HTML DOM can be accessed with JavaScript (and with other programming languages).
- In the DOM, all HTML elements are defined as **objects**.
- The programming interface is the properties and methods of each object.
- A **property** is a value that you can get or set (like changing the content of an HTML element).
- A **method** is an action you can do (like add or deleting an HTML element).

Object tree example



With the object model, JavaScript gets all the power it needs to create dynamic HTML:

- JavaScript can change all the HTML elements in the page
- JavaScript can change all the HTML attributes in the page
- JavaScript can change all the CSS styles in the page
- JavaScript can remove existing HTML elements and attributes
- JavaScript can add new HTML elements and attributes
- JavaScript can react to all existing HTML events in the page
- JavaScript can create new HTML events in the page

The HTML DOM Document Object The document object represents your web page. If you want to access any element in an HTML page, you always start with accessing the document object. Below are some examples of how you can use the document object to access and manipulate HTML.

Finding HTML Elements

Method	Description
<code>document.getElementById(id)</code>	Find an element by element id
<code>document.getElementsByTagName(name)</code>	Find elements by tag name
<code>document.getElementsByClassName(name)</code>	Find elements by class name

Changing HTML Elements

Property	Description
<i>element.innerHTML = new html content</i>	Change the inner HTML of an element
<i>element.attribute = new value</i>	Change the attribute value of an HTML element
<i>element.style.property = new style</i>	Change the style of an HTML element
Method	Description
<i>element.setAttribute(attribute, value)</i>	Change the attribute value of an HTML element

Adding and Deleting Elements

Method	Description
<i>document.createElement(element)</i>	Create an HTML element
<i>document.removeChild(element)</i>	Remove an HTML element
<i>document.appendChild(element)</i>	Add an HTML element
<i>document.replaceChild(new, old)</i>	Replace an HTML element
<i>document.write(text)</i>	Write into the HTML output stream

Changing HTML Content: The easiest way to modify the content of an HTML element is by using the **innerHTML** property. To change the content of an HTML element, use this syntax: `document.getElementById(id).innerHTML = new HTML`

Example: This example changes the content of a **<p>** element:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript can Change HTML</h2>
<p id="p1">Hello World!</p>
<script>
  document.getElementById("p1").innerHTML = "New text!";
</script>
<p>The paragraph above was changed by a script.</p>
</body>
</html>
```

In above example The HTML document above contains a **<p>** element with **id="p1"** We use the HTML DOM to get the element with **id="p1"** A JavaScript changes the content (**innerHTML**) of that element to "New text!"

Changing the Value of an Attribute To change the value of an HTML attribute, use this syntax: `document.getElementById(id).attribute = new value`

```
<!DOCTYPE html>
<html>
<body>

<script>
  document.getElementById("myImage").src = "landscape.jpg";
</script>
</body>
</html>
```

In above example The HTML document above contains an **** element with **id="myImage"**, We use the HTML DOM to get the element with **id="myImage"**. A JavaScript changes the **src** attribute of that element from "smiley.gif" to "landscape.jpg"

Change style:

```
<!DOCTYPE html>
<html>
<body>
<h1>My First JavaScript</h1>
<p id="demo">JavaScript can change the style of an HTML element.</p>
<script>
  function myFunction() {
    document.getElementById("demo").style.fontSize = "25px";
    document.getElementById("demo").style.color = "red";
    document.getElementById("demo").style.backgroundColor =
      "yellow";
  }
</script>
```

```

    <button type="button" onclick="myFunction()">Click Me!</button>
  </body>
</html>

```

Example: Write a javascript to demonstrate create a new html element and add to html page

```

<!DOCTYPE html>
<html>
<body>

```

```

<h1>The Document Object</h1>
<h2>The createElement() Method</h2>

```

```

<p>Create a p element with some text:</p>
<button type="button" onclick="myfun()">create</button>
<script>

```

```

// Create element:

```

```

function myfun()
{
  const para = document.createElement("p");
  para.innerText = "This is a paragraph.";

```

```

// Append to body:
document.body.appendChild(para);
}

```

```

</script>

```

```

</body>
</html>

```

Example: Write a javascript to demonstrate removing a html element from web page

```

<!DOCTYPE html>
<html>
<body>

```

```

<h1>The Element Object</h1>
<h2>The remove() Method</h2>

```

```

<p id="demo">Click "Remove", and this paragraph will be removed from the DOM.</p>
<button onclick="myFunction()">Remove</button>

```

```

<script>
function myFunction() {
  document.getElementById("demo").remove();
}
</script>
</body>
</html>

```

Example: Write a javascript to demonstrate use of checkbox as well as hide and display a html element in html page

```

<!DOCTYPE html>
<html>
<body>

```

```

<p>Display some text when the checkbox is checked:</p>

```

```

<label for="myCheck">Checkbox:</label>
<input type="checkbox" id="myCheck" onclick="myFunction()">

<p id="text" style="display:none">Checkbox is CHECKED!</p>

<script>
function myFunction() {
  var checkBox = document.getElementById("myCheck");
  var text = document.getElementById("text");
  if (checkBox.checked == true){
    text.style.display = "block";
  } else {
    text.style.display = "none";
  }
}
</script>
</body>
</html>

```

Session cookies in JavaScript

Session only cookies, on the other hand, **stores information in the browser memory, and is available for the duration of the browser session.** In other words, the data stored inside a session cookie is available from the time of storage until the browser is closed.

Java script and html Form methods and properties. Java script can be used to access html forms and html input elements.

Javascript forms

The Method Attribute

The **method** attribute specifies the HTTP method to be used when submitting the form data.

The form-data can be sent as URL variables (with **method="get"**) or as HTTP post transaction (with **method="post"**).

The default HTTP method when submitting form data is GET.

```

<!DOCTYPE html>
<html>
<body>
<h2>The method Attribute</h2>
<p>This form will be submitted using the GET method:</p>
<form action="/action_page.php" target="_blank" method="get">
<label for="fname">First name:</label><br>
<input type="text" id="fname" name="fname" value="John"><br>
<label for="lname">Last name:</label><br>
<input type="text" id="lname" name="lname" value="Doe"><br><br>
<input type="submit" value="Submit">
</form>
<p>After you submit, notice that the form values is visible in the address bar of the new browser tab.</p>
</body>
</html>

```

Example

This example uses the GET method when submitting the form data:

```
<form action="/action_page.php" method="get">
```

Example

This example uses the POST method when submitting the form data:

```
<form action="/action_page.php" method="post">
```

Notes on GET:

- Appends the form data to the URL, in name/value pairs
- NEVER use GET to send sensitive data! (the submitted form data is visible in the URL!)
- The length of a URL is limited (2048 characters)
- Useful for form submissions where a user wants to bookmark the result
- GET is good for non-secure data, like query strings in Google

Notes on POST:

- Appends the form data inside the body of the HTTP request (the submitted form data is not shown in the URL)
- POST has no size limitations, and can be used to send large amounts of data.
- Form submissions with POST cannot be bookmarked

onsubmit Event

Example

Execute a JavaScript when a form is submitted:

```
<form onsubmit="myFunction()">
  Enter name: <input type="text">
  <input type="submit">
</form>
```

Output

John

Form is submitted

Definition and Usage The onsubmit event occurs when a form is submitted. Interaction with form elements

Form validation: java script can be used for form validation. Form validation normally used to occur at the server, after the client had entered all the necessary data and then pressed the Submit button. If the data entered by a client was incorrect or was simply missing, the server would have to send all the data back to the client and request that the form be resubmitted with correct information. This was really a lengthy process which used to put a lot of burden on the server.

JavaScript provides a way to validate form's data on the client's computer before sending it to the web server. Form validation generally performs two functions.

- Basic Validation – First of all, the form must be checked to make sure all the mandatory fields are filled in. It would require just a loop through each field in the form and check for data.
- Data Format Validation – Secondly, the data that is entered must be checked for correct form and value. Your code must include appropriate logic to test correctness of data.

Example: Write a to javascript to demonstrate form validation and to validate that text box is not empty

```
<!DOCTYPE html>
<html>
<head>
<script>
function validateForm() {
  let x = document.forms["myForm"]["fname"].value;
  if (x == "") {
    alert("Name must be filled out");
    return false;
  }
}
</script>
</head>
<body>
```

```
<h2>JavaScript Validation</h2>
```

```
<form name="myForm" action="/action_page.php" onsubmit="return validateForm()"
method="post">
  Name: <input type="text" name="fname">
  <input type="submit" value="Submit">
</form>
</body>
</html>
```

Example: Write a javascript to demonstrate form validation and to validate a number value between 1 to 10

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Validation</h2>

<p>Please input a number between 1 and 10:</p>
<input id="numb">
<button type="button" onclick="myFunction()">Submit</button>
<p id="demo"></p>

<script>
function myFunction() {
  // Get the value of the input field with id="numb"
  let x = document.getElementById("numb").value;
  // If x is Not a Number or less than one or greater than 10
  let text;
  if (isNaN(x) || x < 1 || x > 10) {
    text = "Input not valid";
  } else {
    text = "Input OK";
  }
  document.getElementById("demo").innerHTML = text;
}
</script>
</body>
</html>
```

Example: Write a javascript for sum of two number using textbox and button

```
<!doctype html>
<html>
  <head>
    <script>
      function add()
      {
        var num1, num2, sum;
        num1 = parseInt(document.getElementById("firstnumber").value);
        num2 = parseInt(document.getElementById("secondnumber").value);
        sum = num1 + num2;
        document.getElementById("answer").value = sum;
      }
    </script>
  </head>
```

```
<body>
  <p>First Number: <input type="text " id="firstnumber"></p>
  <p>Second Number: <input type="text " id="secondnumber"></p>
  <button onclick="add()">Add Them</button>
  <p>Sum = <input id="answer"></p>
</body>
</html>
```

Web Programming Lab

Programs

Program1: Write a HTML code for printing hello world

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title11</title>
</head>
<body>
<h1>Hello World.....!</h1>
</body>
</html>
```

Program2: Write a HTML code to demonstrate headings, paragraph and different text formats

```
<!DOCTYPE html>
<!-- #program2-->
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>Coolest Document Ever</title>
</head>
<body>
<h1> THIS IS HEADING 1</h1>
<h2> THIS IS HEADING 2</h2>
<h3> THIS IS HEADING 3</h3>
<h4> THIS IS HEADING 4</h4>
<h5> THIS IS HEADING 5</h5>
<h6> THIS IS HEADING 6</h6>
<p>THIS IS A PARAGRAPH. THIS IS A PARAGRAPH. THIS IS A
PARAGRAPH</p>
<p>THIS IS ANOTHER PARAGRAPH. THIS IS ANOTHER PARAGRAPH. THIS IS
ANOTHER PARAGRAPH</p>
<p> THIS IS A LINE BREAK
<br>
THIS IS A LINE BREAK
<br>THIS IS A LINE BREAK</p>
<p> THIS IS AN ORDERED LIST. </p>
<ol>
<li>Item 1</li>
<li>Item 2</li>
<li>Item 3</li>
```

Web Programming Lab Programs

```
</li>Item 4</li>
<li>Item 5</li>
</ol>
<p> THIS IS AN UNORDERED LIST. </p>
<ul>
<li>Item 1</li>
<li>Item 2</li>
<li>Item 3</li>
<li>Item 4</li>
<li>Item 5</li>
<p> <b> THIS TEXT IS BOLD </b> </p>
<p> <i> THIS TEXT IS ITALIC </i></p>
<p> <u> THIS TEXT IS UNDERLINED TEXT </u> </p>
<p> <del> THIS TEXT IS deleted TEXT </del> </p>
<p> <ins> THIS TEXT IS inserted TEXT </ins> </p>
<p> THIS TEXT IS <sup>superscript</sup> </del> </p>
<p> THIS TEXT IS <sub>subscript</sub> </del> </p>
</body>
</html>
```

Program3: Write a HTML to change background color of html page

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title11</title>
</head>
<body bgcolor = "yellow"> or <body bgcolor = "RGB(100,0,0)">
<h1>My First Heading</h1>
<h2>Second Heading</h2>
<p>My first paragraph.</p>
</body>
</html>
```

Program4: Write a HTML to insert Image and Horizontal rule in html page

Note**image name is poster.jpg and stored in same directory of html page

```
<!DOCTYPE html>
<html>
<head>
```

Web Programming Lab Programs

```
<title>image and Hr</title>
</head>
<body>
<h1>example of image</h1>
<img src = "POSTER.JPG" height = "200" width = "200" border=5><br><br>
<h2 style = "color : red">Given below is horizontal rule of size 1</h2>
<hr size="1">
<h2 style = "color : red">Given below is horizontal rule of size 10</h2>
<hr size="10">
<h2 style = "color : red">Given below is horizontal rule of size 1 and size 10 with
noshade </h2>
<hr size="10" noshade>
</body>
</html>
```

Program5: Write a HTML to insert Image in background of html page

Note**image name is poster.jpg and stored in same directory of html page

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title11</title>
</head>
<body background = POSTER.JPG>
<h1>My First Heading</h1>
</body>
</html>
```

Program6: Write a HTML to insert a table to two row and two column inhtml page

```
<!DOCTYPE html>
<html>
<head>
<title>HTML Tables</title>
</head>
<body>
<table border = "1">
<tr>
<th>Row 1, Column 1</th>
<th>Row 1, Column 2</th>
</tr>
```

Web Programming Lab Programs

```
<tr>
  <td>Row 2, Column 1</td>
  <td>Row 2, Column 2</td>
</tr>
</table>
</body>
</html>
```

Program7: Write a HTML to insert a textbox and button in form html page

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>HTML Forms</h2>
```

```
<form action="/action_page.php">
```

```
  <label for="fname">First name:</label><br>
```

```
  <input type="text" id="fname" name="fname" value="John"><br>
```

```
  <label for="lname">Last name:</label><br>
```

```
  <input type="text" id="lname" name="lname" value="Doe"><br><br>
```

```
  <input type="submit" value="Submit">
```

```
</form>
```

```
<p>If you click the "Submit" button, the form-data will be sent to a page called  
"/action_page.php".</p>
```

```
</body>
```

```
</html>
```

Program8: Write a HTML to insert a form and checkbox in html page

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>Show Checkboxes</h1>
```

```
<form action="/action_page.php">
```

```
  <input type="checkbox" id="vehicle1" name="vehicle1" value="Bike">
```

```
  <label for="vehicle1"> I have a bike</label><br>
```

```
  <input type="checkbox" id="vehicle2" name="vehicle2" value="Car">
```

```
  <label for="vehicle2"> I have a car</label><br>
```

```
  <input type="checkbox" id="vehicle3" name="vehicle3" value="Boat">
```

Web Programming Lab Programs

```
<label for="vehicle3"> I have a boat</label><br><br>
<input type="submit" value="Submit">
</form>
```

```
</body>
</html>
```

Program9: Write a HTML code to insert a form and select element in html page

```
<!DOCTYPE html>
<html>
<body>
<h1>Show select element</h1>
<form action="/action_page.php">
  <label for="cars">Choose a car:</label>
  <select id="cars" name="cars">
    <option value="volvo">Volvo</option>
    <option value="saab">Saab</option>
    <option value="fiat">Fiat</option>
    <option value="audi">Audi</option>
  </select>
  <input type="submit">
</form>
```

```
</body>
</html>
```

Program10: Write a html code to demonstrate inline CSS for changing color of heading

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title11</title>
</head>
<body>

<h1 style = "color : red">Color change inline CSS</h1>

</body>
</html>
```

Web Programming Lab Programs

Program11: Write a html code to demonstrate CSS by external file for changing colors of background and heading

HTML CODE: Write in .html file

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="mystyle.css">
</head>
<body>
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
</body>
</html>
```

CSS CODE: Write in .css file with file name mystyle.ss

```
body {
  background-color: yellow;
}

h1 {
  color: navy;
  margin-left: 20px;
}
```

Program12: Write a javascript to insert/change content of html element

```
<!DOCTYPE html>
<html>
<body>
<h2>What Can JavaScript Do?</h2>

<p id="demo">JavaScript can change HTML content.</p>

<button type="button" onclick='document.getElementById("demo").innerHTML =
"Hello JavaScript!">Click Me!</button>

</body>
</html>
```

Program13: Write a javascript to demonstrate use of checkbox as well as hide and display a html element in html page

```
<!DOCTYPE html>
<html>
<body>

<p>Display some text when the checkbox is checked:</p>

<label for="myCheck">Checkbox:</label>
<input type="checkbox" id="myCheck" onclick="myFunction()">
```

Web Programming Lab Programs

```
<p id="text" style="display:none">Checkbox is CHECKED!</p>
```

```
<script>
function myFunction() {
  var checkBox = document.getElementById("myCheck");
  var text = document.getElementById("text");
  if (checkBox.checked == true){
    text.style.display = "block";
  } else {
    text.style.display = "none";
  }
}
</script>
```

```
</body>
```

```
</html>
```

Program14: Write a javascript for sum of two number using textbox and button

```
<!doctype html>
```

```
<html>
```

```
  <head>
```

```
    <script>
```

```
      function add()
```

```
      {
```

```
        var num1, num2, sum;
```

```
        num1 = parseInt(document.getElementById("firstnumber").value);
```

```
        num2 = parseInt(document.getElementById("secondnumber").value);
```

```
        sum = num1 + num2;
```

```
        document.getElementById("answer").value = sum;
```

```
      }
```

```
    </script>
```

```
  </head>
```

```
  <body>
```

```
    <p>First Number: <input type="text " id="firstnumber"></p>
```

```
    <p>Second Number: <input type="text " id="secondnumber"></p>
```

```
    <button onclick="add()">Add Them</button>
```

```
    <p>Sum = <input id="answer"></p>
```

```
  </body>
```

```
</html>
```

Web Programming Lab Programs

Program15: Write a javascript to demonstrate form validation and to validate that text box is not empty

```
<!DOCTYPE html>
<html>
<head>
<script>
function validateForm() {
  let x = document.forms["myForm"]["fname"].value;
  if (x == "") {
    alert("Name must be filled out");
    return false;
  }
}
</script>
</head>
<body>

<h2>JavaScript Validation</h2>

<form name="myForm" action="/action_page.php" onsubmit="return validateForm()"
method="post">
  Name: <input type="text" name="fname">
  <input type="submit" value="Submit">
</form>
</body>
</html>
```

Program16: Write a javascript to demonstrate form validation and to validate a number value between 1 to 10

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Validation</h2>

<p>Please input a number between 1 and 10:</p>
<input id="numb">
<button type="button" onclick="myFunction()">Submit</button>
<p id="demo"></p>
```

Web Programming Lab Programs

```
<script>
function myFunction() {
  // Get the value of the input field with id="numb"
  let x = document.getElementById("numb").value;
  // If x is Not a Number or less than one or greater than 10
  let text;
  if (isNaN(x) || x < 1 || x > 10) {
    text = "Input not valid";
  } else {
    text = "Input OK";
  }
  document.getElementById("demo").innerHTML = text;
}
</script>
</body>
</html>
```

Program17: Write a javascript to demonstrate create a new html element and add to html page

```
<!DOCTYPE html>
<html>
<body>
```

```
<h1>The Document Object</h1>
<h2>The createElement() Method</h2>
```

```
<p>Create a p element with some text:</p>
<button type="button" onclick="myfun()">create</button>
```

```
<script>
// Create element:
function myfun()
{
  const para = document.createElement("p");
  para.innerText = "This is a paragraph.";
```

```
  // Append to body:
  document.body.appendChild(para);
}
```

```
</script>
```

Web Programming Lab Programs

```
</body>
```

```
</html>
```

Program18: Write a javascript to demonstrate removing a html element from web page

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>The Element Object</h1>
```

```
<h2>The remove() Method</h2>
```

```
<p id="demo">Click "Remove", and this paragraph will be removed from the DOM.</p>
```

```
<button onclick="myFunction()">Remove</button>
```

```
<script>
```

```
function myFunction() {
```

```
    document.getElementById("demo").remove();
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

PHP started out as a small open source project that evolved as more and more people found out how useful it was. Rasmus Lerdorf unleashed the first version of PHP way back in 1994.

- PHP is a recursive acronym for "PHP: Hypertext Preprocessor".
- PHP is a server side scripting language that is embedded in HTML. It is used to manage dynamic content, databases, session tracking, even build entire e-commerce sites.
- It is integrated with a number of popular databases, including MySQL, PostgreSQL, Oracle, Sybase, Informix, and Microsoft SQL Server.
- PHP is pleasingly zippy in its execution, especially when compiled as an Apache server module on the Unix side. The MySQL server, once started, executes even very complex queries with huge result sets in record-setting time.
- PHP supports a large number of major protocols such as POP3, IMAP, and LDAP. PHP4 added support for Java and distributed object architectures (COM and CORBA), making n-tier development a possibility for the first time.
- PHP Syntax is C-Like. So its simple.

Common Uses of PHP

PHP performs system functions, i.e. from files on a system it can create, open, read, write, and close them. The other uses of PHP are:

- It can generate dynamic page
- PHP can handle forms, i.e. gather data from files, save data to a file, thru email you can send data, return data to the user.
- You add, delete, modify elements within your database thru PHP.
- Access cookies variables and set cookies.
- Using PHP, you can restrict users to access some pages of your website.
- By PHP, we can create,open,close,read,write a file on server
- It can encrypt data.
- Output can be given on HTML,XHTML,XML,PDF

Characteristics of PHP

Five important characteristics make PHP's practical nature possible:

- Simplicity: Syntax is as simple as C
- Efficiency: Fast and light weight
- Security: Support security features
- Flexibility: support multiple platforms like window,unix,linux,ios. Compitible with almost all server technology, support wide range of database. Supports all market protocols like POP3,IMAP,LDAP. Support java and distributed architecture like COBRA, COM
- Familiarity

NEEDS FOR USINH PHP

In order to develop and run PHP Web pages, three vital components need to be installed on your computer system.

Web Server - PHP will work with virtually all **Web Server** software, including Microsoft's Internet Information Server (IIS) but then most often used is freely available Apache Server.

Download Apache for free here: <http://httpd.apache.org/download.cgi>

Database - PHP will work with virtually all database software, including Oracle and Sybase but most commonly used is freely available MySQL database. Download MySQL for free here: <http://www.mysql.com/downloads/index.html>

PHP Parser - In order to process PHP script instructions, a parser must be installed to generate HTML output that can be sent to the **Web Browser**. This tutorial will guide you how to install PHP parser on your computer.

Before you proceed, it is important to make sure that you have a proper environment setup on your machine to develop your web programs using PHP.

Type the following address into your browser's address box.

`http://127.0.0.1/info.php`

If this displays a page showing your PHP installation related information, then it means you have PHP and Webserver installed properly. Otherwise you have to follow given procedure to install PHP on your computer.

"Hello World" Script in PHP

To get a feel of PHP, first start with simple PHP scripts. Since "Hello, World!" is an essential example, first we will create a friendly little "Hello, World!" script.

As mentioned earlier, PHP is embedded in HTML. That means that in amongst your normal HTML (or XHTML if you're cutting-edge) you'll have PHP statements like this:

```
<html>
<head>
<title>Hello World</title>
<body>
    <?php echo "Hello, World!";?>
</body>
</html>
```

It will produce the following result:

Hello, World!

If you examine the HTML output of the above example, you'll notice that the PHP code is not present in the file sent from the server to your Web browser. All of the PHP present in the Web page is processed and stripped from the page; the only thing returned to the client from the Web server is pure HTML output.

All PHP code must be included inside one of the three special markup tags that are recognized by the PHP Parser.

```
<?php PHP code goes here ?>

<?    PHP code goes here ?>

<script language="php"> PHP code goes here </script>
```

Most common tag is the `<?php...?>` and we will also use the same tag in our DOCUMENT

Escaping to PHP

The PHP parsing engine needs a way to differentiate PHP code from other elements in the page. The mechanism for doing so is known as 'escaping to PHP.' There are four ways to do this:

Canonical PHP tags: The most universally effective PHP tag style is:

```
<?php...?>
```

Short-open (SGML-style) tags:

Short or short-open tags look like this:

```
<?...?>
```

ASP-style tags

ASP-style tags mimic the tags used by Active Server Pages to delineate code blocks. ASP-style tags look like this:

```
<%...%>
```

HTML script tags

HTML script tags look like this:

```
<script language="PHP">...</script>
```

Commenting PHP Code

A *comment* is the portion of a program that exists only for the human reader and stripped out before displaying the programs result. There are two commenting formats in PHP:

Single-line comments: They are generally used for short explanations or notes relevant to the local code. Here are the examples of single line comments.

```
<?
# This is a comment, and
# This is the second line of the comment
// This is a comment too. Each style comments only
echo "An example with single line comments";
/*this is a multi line comment
line 1
Line2*/
?>
```

Multi-lines printing: Here are the examples to print multiple lines in a single print statement:

```
<?
# First Example
print <<<END
This uses the "here document" syntax to output
multiple lines with $variable interpolation. Note
that the here document terminator must appear on a
line with just a semicolon no extra whitespace!
END;
# Second Example
print "This spans
multiple lines. The newlines will be
output as well";
?>
```

PHP is whitespace insensitive

Whitespace is the stuff you type that is typically invisible on the screen, including spaces, tabs, and carriage returns (end-of-line characters).

PHP whitespace insensitive means that it almost never matters how many whitespace characters you have in a row. one whitespace character is the same as many such characters. For example, each of the following PHP statements that assigns the sum of 2 + 2 to the variable \$four is equivalent:

```
$four = 2 + 2; // single spaces
$four <tab>=<tab>2<tab>+<tab>2 ; // spaces and tabs
$four =
2+
2; // multiple lines
```

PHP is case sensitive

Yes it is true that PHP is a case sensitive language. But keywords are not case sensitive.

Statements are expressions terminated by semicolons

A *statement* in PHP is any expression that is followed by a semicolon (;). Any sequence of valid PHP statements that is enclosed by the PHP tags is a valid PHP program. Here is a typical statement in PHP, which in this case assigns a string of characters to a variable called \$greeting:

```
$greeting = "Welcome to PHP!";
```

Variables:

The main way to store information in the middle of a PHP program is by using a variable. Here are the most important things to know about variables in PHP.

- All variables in PHP are denoted with a leading dollar sign (\$).
- The value of a variable is the value of its most recent assignment.
- Variables are assigned with the = operator, with the variable on the left-hand side and the expression to be evaluated on the right.
- Variables can, but do not need, to be declared before assignment.

- **Php is loosely typed language:** Variables in PHP do not have intrinsic types - a variable does not know in advance whether it will be used to store a number or a string of characters.

A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume). Rules for PHP variables:

- A variable starts with the \$ sign, followed by the name of the variable
- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (\$age and \$AGE are two different variables)

Examples:

```
<!DOCTYPE html>
<html>
<body>
<?php
$txt = "KDU";
echo "I love " . $txt . "!";
?>
</body>
</html>
Output: I love KDU
```

```
<!DOCTYPE html>
<html>
<body>
<?php
$x = 5;
$y = 4;
echo $x + $y;
?>
</body></html>
Output: 9
```

Data types:

- **Integers:** are whole numbers, without a decimal point, like 4195. An integer data type is a non-decimal number between -2,147,483,648 and 2,147,483,647.
- **Doubles:** are floating-point numbers, like 3.14159 or 49.1.
- **Booleans:** have only two possible values either true or false.
- **NULL:** is a special type that only has one value: NULL.
- **Strings:** are sequences of characters, like 'PHP supports string operations.'
- **Arrays:** are named and indexed collections of other values.
- **Objects:** are instances of programmer-defined classes, which can package up both other kinds of values and functions that are specific to the class.
- **Resources:** are special variables that hold references to resources external to PHP (such as database connections).

The first five are *simple types*, and the next two (arrays and objects) are compound - the compound types can package up other arbitrary values of arbitrary type, whereas the simple types cannot. Example: variable and var_dump() function to check types of data type

```
<!DOCTYPE html>
<html>
<body>
<?php
$x = 5985;
var_dump($x);
?>
</body>
</html>
```

Output: int(5985)

```
<!DOCTYPE html>
<html>
<body>
<?php
$cars =
array("Volvo","BMW","Toyota");
var_dump($cars);
?>
</body>
</html>
```

Output: array(3) {
 [0]=> string(5) "Volvo"
 [1]=> string(3) "BMW"
 [2]=> string(6) "Toyota" }

PHP – Scope of variables

Scope can be defined as the range of availability a variable has to the program in which it is declared. PHP variables can be one of four scope types:

- Local variables
- Global variables
- Static variables

PHP Local Variables

A variable declared in a function is considered local; that is, it can be referenced solely in that function. Any assignment outside of that function will be considered to be an entirely different variable from the one contained in the function:

PHP Global Variables

In contrast to local variables, a global variable can be accessed in any part of the program. However, in order to be modified, a global variable must be explicitly declared to be global in the function in which it is to be modified. This is accomplished, conveniently enough, by placing the keyword **GLOBAL** in front of the variable that should be recognized as global. Placing this keyword in front of an already existing variable tells PHP to use the variable having that name.

Global keyword and array: The `global` keyword is used to access a global variable from within a function. To do this, use the `global` keyword before the variables (inside the function):

```
<!DOCTYPE html>
<html>
<body>
<?php
$x = 5;
$y = 10; //x,y defined outside the function
function myTest() {
    global $x, $y;
    $y = $x + $y;
}
myTest(); // run function
echo $y; // output 15
?>
</body>
</html>
```

Global array: PHP also stores all global variables in an array called `$GLOBALS[index]`. The *index* holds the name of the variable. This array is also accessible from within functions and can be used to update global variables directly.

Example: The above example can be used like this

```
<!DOCTYPE html>
<html>
<body>
<?php
$x = 5;
$y = 10;
function myTest() {
    $GLOBALS['y'] = $GLOBALS['x'] + $GLOBALS['y'];
}
myTest();
echo $y;
?>
</body>
</html>
```

PHP Static Variables

The final type of variable scoping that I discuss is known as static. In contrast to the variables declared as function parameters, which are destroyed on the function's exit, a static variable will not lose its value when the function exits and will still hold that value should the function be called again.

You can declare a variable to be static simply by placing the keyword `STATIC` in front of the variable name.

Example:

```
<!DOCTYPE html>
<html>
<body>
<?php
function myTest() {
    static $x = 0;
    echo $x;
    $x++;
}
myTest();
echo "<br>";
myTest();
echo "<br>";
myTest();
?>
</body>
</html>
```

The output is :

```
0
1
2
```

PHP Constants:

PHP

A constant is a name or an identifier for a simple value. A constant value cannot change during the execution of the script. By default, a constant is case-sensitive. By convention, constant identifiers are always uppercase. A constant name starts with a letter or underscore, followed by any number of letters, numbers, or underscores. If you have defined a constant, it can never be changed or undefined.

To define a constant you have to use `define()` function and to retrieve the value of a constant, you have to simply specifying its name. Unlike with variables, you do not need to have a constant with a \$.

Syntax: `define(name, value, case-insensitive)`

- *name*: Specifies the name of the constant
- *value*: Specifies the value of the constant
- *case-insensitive*: Specifies whether the constant name should be case-insensitive. Default is false

Example:

```
<?php
// case-sensitive constant name
define("GREETING", "Welcome to W3Schools.com!");
echo GREETING;
?>
```

PHP special constants: PHP provides a large number of predefined constants to any script which it runs. There are five magical constants that change depending on where they are used. For example, the value of `_LINE` depends on the line that it's used on in your script. These special constants are case-insensitive and are as follows:

PHP Operator? Simple answer can be given using expression *4 + 5 is equal to 9*. Here 4 and 5 are called operands and + is called operator. PHP language supports following type of operators.

Arithmetic Operator	Name	Example	Result
+	Addition	$\$x + \y	Sum of \$x and \$y
-	Subtraction	$\$x - \y	Difference of \$x and \$y
*	Multiplication	$\$x * \y	Product of \$x and \$y
/	Division	$\$x / \y	Quotient of \$x and \$y
%	Modulus	$\$x \% \y	Remainder of \$x divided by \$y
**	Exponentiation	$\$x ** \y	Result of raising \$x to the \$y'th power

Assignment	Same as...	Description
<code>x = y</code>	<code>x = y</code>	The left operand gets set to the value of the expression on the right
<code>x += y</code>	<code>x = x + y</code>	Addition
<code>x -= y</code>	<code>x = x - y</code>	Subtraction
<code>x *= y</code>	<code>x = x * y</code>	Multiplication
<code>x /= y</code>	<code>x = x / y</code>	Division
<code>x %= y</code>	<code>x = x % y</code>	Modulus

Comparison

Operator	Name	Example	Result
<code>==</code>	Equal	<code>\$x == \$y</code>	Returns true if \$x is equal to \$y
<code>===</code>	Identical	<code>\$x === \$y</code>	Returns true if \$x is equal to \$y, and they are of the same type
<code>!=</code>	Not equal	<code>\$x != \$y</code>	Returns true if \$x is not equal to \$y
<code><></code>	Not equal	<code>\$x <> \$y</code>	Returns true if \$x is not equal to \$y
<code>!==</code>	Not identical	<code>\$x !== \$y</code>	Returns true if \$x is not equal to \$y, or they are not of the same type
<code>></code>	Greater than	<code>\$x > \$y</code>	Returns true if \$x is greater than \$y
<code><</code>	Less than	<code>\$x < \$y</code>	Returns true if \$x is less than \$y
<code>>=</code>	Greater than or equal to	<code>\$x >= \$y</code>	Returns true if \$x is greater than or equal to \$y
<code><=</code>	Less than or equal to	<code>\$x <= \$y</code>	Returns true if \$x is less than or equal to \$y
<code><=></code>	Spaceship	<code>\$x <=> \$y</code>	Returns an integer less than, equal to, or greater than zero, depending on if \$x is less than, equal to, or greater than \$y. Introduced in PHP 7.

Increment/decrement

Operator	Name	Description
<code>++\$x</code>	Pre-increment	Increments <code>\$x</code> by one, then returns <code>\$x</code>
<code>\$x++</code>	Post-increment	Returns <code>\$x</code> , then increments <code>\$x</code> by one
<code>--\$x</code>	Pre-decrement	Decrements <code>\$x</code> by one, then returns <code>\$x</code>
<code>\$x--</code>	Post-decrement	Returns <code>\$x</code> , then decrements <code>\$x</code> by one

String operator

Operator	Name	Example	Result	Show it
<code>.</code>	Concatenation	<code>\$txt1 . \$txt2</code>	Concatenation of <code>\$txt1</code> and <code>\$txt2</code>	
<code>.=</code>	Concatenation assignment	<code>\$txt1 .= \$txt2</code>	Appends <code>\$txt2</code> to <code>\$txt1</code>	

Conditional operator

Operator	Name	Example	Result	Show it
<code>?:</code>	Ternary	<code>\$x = <i>expr1</i> ? <i>expr2</i> : <i>expr3</i></code>	Returns the value of <code>\$x</code> . The value of <code>\$x</code> is <code><i>expr2</i></code> if <code><i>expr1</i></code> = TRUE. The value of <code>\$x</code> is <code><i>expr3</i></code> if <code><i>expr1</i></code> = FALSE	
<code>??</code>	Null coalescing	<code>\$x = <i>expr1</i> ?? <i>expr2</i></code>	Returns the value of <code>\$x</code> . The value of <code>\$x</code> is <code><i>expr1</i></code> if <code><i>expr1</i></code> exists, and is not NULL. If <code><i>expr1</i></code> does not exist, or is NULL, the value of <code>\$x</code> is <code><i>expr2</i></code> . Introduced in PHP 7	

PHP Conditional Statements: Very often when you write code, you want to perform different actions for different conditions. You can use conditional statements in your code to do this. In PHP we have the following conditional statements (all are same C++/java):

- **if** statement - executes some code if one condition is true
- **if...else** statement - executes some code if a condition is true and another code if that condition is false
- **if...elseif...else** statement - executes different codes for more than two conditions

- **switch** statement - selects one of many blocks of code to be executed.

```
<?php
$t = date("H");

if ($t < "20") {
    echo "Have a good day!";
} else {
    echo "Have a good
night!";
}
?>
```

```
<?php
$t = date("H");

if ($t < "10") {
    echo "Have a good
morning!";
} elseif ($t < "20") {
    echo "Have a good day!";
} else {
    echo "Have a good
night!";
}
?>
```

```
<?php
$favcolor = "red";

switch ($favcolor) {
    case "red":
        echo "Your favorite
color is red!";
        break;
    case "blue":
        echo "Your favorite
color is blue!";
        break;
    case "green":
        echo "Your favorite
color is green!";
        break;
    default:
        echo "Your favorite
color is neither red, blue,
nor green!";
}
?>
```

PHP Loops

Often when you write code, you want the same block of code to run over and over again a certain number of times. So, instead of adding several almost equal code-lines in a script, we can use loops. Loops are used to execute the same block of code again and again, as long as a certain condition is true. In PHP, we have the following loop types:

- **while** - loops through a block of code as long as the specified condition is true
- **do...while** - loops through a block of code once, and then repeats the loop as long as the specified condition is true
- **for** - loops through a block of code a specified number of times
- **foreach** - loops through a block of code for each element in an array

The following chapters will explain and give examples of each loop type. The syntax is same as c/c++ for first 3 loops.

```
while (condition is true)
{
    code to be executed;
}
```

```
do {
    code to be executed;
} while (condition is
true);
```

```
for (init counter; test counter; increment counter) {
    code to be executed for each iteration;
}
```

For Each loop: The **foreach** loop works only on arrays, and is used to loop through each key/value pair in an array. **Syntax is given below**

```
foreach ($array as $value) {
    code to be executed;
}
```

For every loop iteration, the value of the current array element is assigned to \$value and the array pointer is moved by one, until it reaches the last array element.

Examples The following example will output the values of the given array (\$colors):

```
<?php
$colors = array("red", "green", "blue", "yellow");
foreach ($colors as $value) {
    echo "$value <br>";
}
?>
```

Example: The following example will output both the keys and the values of the given array (\$age):

```
<?php
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");
foreach($age as $x => $val) {
    echo "$x = $val<br>";
}
?>
```

PHP BREAK: we have already seen the **break** statement used in an earlier chapter of this tutorial. It was used to "jump out" of a **switch** statement. The **break** statement can also be used to jump out of a loop.

Continue: The **continue** statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

Functions: The real power of PHP comes from its functions. PHP has more than 1000 built-in functions, and in addition you can create your own custom functions.

PHP Built-in Functions: **PHP has over 1000 built-in functions that can be called directly, from within a script, to perform a specific task.**

PHP User Defined Functions: **Besides the built-in PHP functions, it is possible to create your own functions.**

- **A function is a block of statements that can be used repeatedly in a program.**
- **A function will not execute automatically when a page loads.**
- **A function will be executed by a call to the function.**

Create a User Defined Function in PHP: A user-defined function declaration starts with the word **function**:

Syntax

```
function functionName() {  
    code to be executed;  
}
```

Note: A function name must start with a letter or an underscore. Function names are NOT case-sensitive.

Tip: Give the function a name that reflects what the function does!

In the example below, we create a function named "writeMsg()". The opening curly brace ({) indicates the beginning of the function code, and the closing curly brace (}) indicates the end of the function. The function outputs "Hello world!". To call the function, just write its name followed by brackets ():

PHP Function Arguments: Information can be passed to functions through arguments. An argument is just like a variable. Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

The following example has a function with one argument (\$fname). When the familyName() function is called, we also pass along a name (e.g. Jani), and the name is used inside the function, which outputs several different first names, but an equal last name:

```
<?php  
function familyName($fname) {  
    echo "$fname Refsnes.<br>";  
}  
familyName("Jani");  
familyName("Hege");  
familyName("Stale");  
familyName("Kai Jim");  
familyName("Borge"); ?>
```

In the example above, notice that we did not have to tell PHP which data type the variable is.

PHP automatically associates a data type to the variable, depending on its value. Since the data types are not set in a strict sense, you can do things like adding a string to an integer without causing an error.

In PHP 7, type declarations were added. This gives us an option to specify the expected data type when declaring a function, and by adding the `strict` declaration, it will throw a "Fatal Error" if the data type mismatches. In the following example we try to send both a number and a string to the function without using `strict`:

Example sum of two numbers

```
<?php
function addNumbers(int $a, int $b) {
    return $a + $b;
}
echo addNumbers(5, "5 days");
// since strict is NOT enabled "5 days" is changed to int(5), and it will
return 10
?>
```

To specify `strict` we need to set `declare(strict_types=1);`. This must be on the very first line of the PHP file. In the following example we try to send both a number and a string to the function, but here we have added the `strict` declaration:

```
<?php declare(strict_types=1); // strict requirement

function addNumbers(int $a, int $b) {
    return $a + $b;
}
echo addNumbers(5, "5 days");
// since strict is enabled and "5 days" is not an integer, an error will be thrown
?>
```

PHP Default Argument Value

The following example shows how to use a default parameter. If we call the function `setHeight()` without arguments it takes the default value as argument:

```
<?php declare(strict_types=1); // strict requirement
function setHeight(int $minheight = 50) {
    echo "The height is : $minheight <br>";
}
setHeight(350);
setHeight(); // will use the default value of 50
setHeight(135);
setHeight(80); ?>
```

Example of strict with function returning value

```
<?php declare(strict_types=1); // strict requirement
function sum(int $x, int $y) {
    $z = $x + $y;
    return $z;
}
echo "5 + 10 = " . sum(5, 10) . "<br>";
echo "7 + 13 = " . sum(7, 13) . "<br>";
echo "2 + 4 = " . sum(2, 4);
?>
```

Passing Arguments by Reference

In PHP, arguments are usually passed by value, which means that a copy of the value is used in the function and the variable that was passed into the function cannot be changed. When a function argument is passed by reference, changes to the argument also change the variable that was passed in. To turn a function argument into a reference, the **&** operator is used:

```
<?php
function add_five(&$value) {
    $value += 5;
}
$num = 2;
add_five($num);
echo $num; //output is 7
?>
```

Arrays

An array stores multiple values in one single variable: An array is a special variable, which can hold more than one value at a time. An array can hold many values under a single name, and you can access the values by referring to an index number. In PHP, the **array()** function is used to create an array: `array();`

In PHP, there are three types of arrays:

- **Indexed arrays** - Arrays with a numeric index
- **Associative arrays** - Arrays with named keys
- **Multidimensional arrays** - Arrays containing one or more arrays

PHP Indexed Arrays There are two ways to create indexed arrays, The index can be assigned automatically (index always starts at 0) or the index can be assigned manually: The following example creates an indexed array named \$cars, assigns three elements to it, and then prints a text containing the array values:

```
<?php
$cars = array("Volvo", "BMW", "Toyota");
$arrlength = count($cars);
for($x = 0; $x < $arrlength; $x++) {
    echo $cars[$x];
    echo "<br>";
}
?>
```

PHP Associative Arrays

Associative arrays are arrays that use named keys that you assign to them. There are two ways to create an associative array:

```
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");
```

or:

```
$age['Peter'] = "35";
```

```
$age['Ben'] = "37";
```

```
$age['Joe'] = "43";
```

The named keys can then be used in a script: Example:

```
<!DOCTYPE html>
<html>
<body>
<?php
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");
foreach($age as $x => $x_value) {
    echo "Key=" . $x . ", Value=" . $x_value;
    echo "<br>";
}
?>
</body>
</html>
```

PHP - Multidimensional Arrays

A multidimensional array is an array containing one or more arrays. PHP supports multidimensional arrays that are two, three, four, five, or more levels deep. However, arrays more than three levels deep are hard to manage for most people.

The dimension of an array indicates the number of indices you need to select an element.

- For a two-dimensional array you need two indices to select an element
- For a three-dimensional array you need three indices to select an element

```
<?php
for ($row = 0; $row < 4; $row++) {
    echo "<p><b>Row number $row</b></p>";
    echo "<ul>";
    for ($col = 0; $col < 3; $col++) {
        echo "<li>".$cars[$row][$col]."</li>";
    }
    echo "</ul>"; } ?>
```

Example: multidimensional associative array

```
<html>
<body>

<?php
    $marks = array(
        "mohammad" => array (
            "physics" => 35,
            "maths" => 30,
            "chemistry" => 39
        ),

        "qadir" => array (
            "physics" => 30,
            "maths" => 32,
            "chemistry" => 29
        ),

        "zara" => array (
            "physics" => 31,
            "maths" => 22,
            "chemistry" => 39
        )
    );

    /* Accessing multi-dimensional array values */
    echo "Marks for mohammad in physics : " ;
    echo $marks['mohammad']['physics'] . "<br />";

    echo "Marks for qadir in maths : ";
    echo $marks['qadir']['maths'] . "<br />";

    echo "Marks for zara in chemistry : " ;
    echo $marks['zara']['chemistry'] . "<br />";
?>

</body>
</html>
```

PHP - Sort Functions For Arrays

In this chapter, we will go through the following PHP array sort functions:

- **sort()** - sort arrays in ascending order
- **rsort()** - sort arrays in descending order
- **asort()** - sort associative arrays in ascending order, according to the value
- **ksort()** - sort associative arrays in ascending order, according to the key
- **arsort()** - sort associative arrays in descending order, according to the value
- **krsort()** - sort associative arrays in descending order, according to the key

```

<!DOCTYPE html>
<html>
<body>
<?php
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");
ksort($age);
foreach($age as $x => $x_value) {
    echo "Key=" . $x . ", Value=" . $x_value;
    echo "<br>";
}
?>
</body>
</html>

```

Output:

```

Key=Ben, Value=37
Key=Joe, Value=43
Key=Peter, Value=35

```

PHP MySQL Database

With PHP, you can connect to and manipulate databases. MySQL is the most popular database system used with PHP.

What is MySQL?

- MySQL is a database system used on the web
- MySQL is a database system that runs on a server
- MySQL is ideal for both small and large applications
- MySQL is very fast, reliable, and easy to use
- MySQL uses standard SQL
- MySQL compiles on a number of platforms
- MySQL is free to download and use
- MySQL is developed, distributed, and supported by Oracle Corporation
- MySQL is named after co-founder Monty Widenius's daughter: My

The data in a MySQL database are stored in tables. A table is a collection of related data, and it consists of columns and rows. Databases are useful for storing information categorically. A company may have a database with the following tables:

- Employees
- Products
- Customers
- Orders

PHP + MySQL Database System : PHP combined with MySQL are cross-platform (you can develop in Windows and serve on a Unix platform)

Database Queries: A query is a question or a request. We can query a database for specific information and have a recordset returned.

Look at the following query (using standard SQL):

```
SELECT LastName FROM Employees
```

The query above selects all the data in the "LastName" column from the "Employees" table.

MySQL is the de-facto standard database system for web sites with HUGE volumes of both data and end-users (like Facebook, Twitter, and Wikipedia). Another great thing about MySQL is that it can be scaled down to support embedded database applications.

Open a Connection to MySQL

Before we can access data in the MySQL database, we need to be able to connect to the server:

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
// Create connection
$conn = new mysqli($servername, $username, $password);
// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
echo "Connected successfully";
?>
```

Create a MySQL Database

A database consists of one or more tables. You will need special CREATE privileges to create or to delete a MySQL database. The CREATE DATABASE statement is used to create a database in MySQL. The following examples create a database named "myDB":

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
// Create connection
$conn = new mysqli($servername, $username, $password);
// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
// Create database
$sql = "CREATE DATABASE myDB";
```

```

if ($conn->query($sql) === TRUE) {
    echo "Database created successfully";
} else {
    echo "Error creating database: " . $conn->error;
}
$conn->close(); //after finishing work we have to close the connection
?>

```

Note: When you create a new database, you must only specify the first three arguments to the mysqli object (servername, username and password).

Tip: If you have to use a specific port, add an empty string for the database-name argument, like this: new mysqli("localhost", "username", "password", "", port)

Create a MySQL Table Using MySQLi

A database table has its own unique name and consists of columns and rows. The CREATE TABLE statement is used to create a table in MySQL. We will create a table named "MyGuests", with five columns: "id", "firstname", "lastname", "email" and "reg_date":

```

CREATE TABLE MyGuests (
id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
firstname VARCHAR(30) NOT NULL,
lastname VARCHAR(30) NOT NULL,
email VARCHAR(50),
reg_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
)

```

Notes on the table above:

The data type specifies what type of data the column can hold. For a complete reference of all the available data types, go to our [Data Types reference](#).

After the data type, you can specify other optional attributes for each column:

- NOT NULL - Each row must contain a value for that column, null values are not allowed
- DEFAULT value - Set a default value that is added when no other value is passed
- UNSIGNED - Used for number types, limits the stored data to positive numbers and zero
- AUTO INCREMENT - MySQL automatically increases the value of the field by 1 each time a new record is added
- PRIMARY KEY - Used to uniquely identify the rows in a table. The column with PRIMARY KEY setting is often an ID number, and is often used with AUTO_INCREMENT

Each table should have a primary key column (in this case: the "id" column). Its value must be unique for each record in the table.

The following examples shows how to create the table in PHP:

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";

// Create connection
//note**: If after creating database you closed connection, always you have to open a connection
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// sql to create table
$sql = "CREATE TABLE MyGuests (
id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
firstname VARCHAR(30) NOT NULL,
lastname VARCHAR(30) NOT NULL,
email VARCHAR(50),
reg_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
)";

if ($conn->query($sql) === TRUE) {
    echo "Table MyGuests created successfully";
} else {
    echo "Error creating table: " . $conn->error;
}
$conn->close(); // connection close
?>
```

Insert Data Into MySQL

After a database and a table have been created, we can start adding data in them. Here are some syntax rules to follow:

- The SQL query must be quoted in PHP
- String values inside the SQL query must be quoted
- Numeric values must not be quoted
- The word NULL must not be quoted

The INSERT INTO statement is used to add new records to a MySQL table:

```
INSERT INTO table_name (column1, column2, column3,...)
VALUES (value1, value2, value3,...)
```

In the previous topic we created an empty table named "MyGuests" with five columns: "id", "firstname", "lastname", "email" and "reg_date". Now, let us fill the table with data.

Note: If a column is AUTO_INCREMENT (like the "id" column) or TIMESTAMP with default update of current_timestamp (like the "reg_date" column), it is no need to be specified in the SQL query; MySQL will automatically add the value.

The following examples add a new record to the "MyGuests" table:

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

$sql = "INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('John', 'Doe', 'john@example.com')";

if ($conn->query($sql) === TRUE) {
    echo "New record created successfully";
} else {
    echo "Error: " . $sql . "<br>" . $conn->error;
}

$conn->close();
?>
```

Select and Filter Data With MySQLi

The following example selects the id, firstname and lastname columns from the MyGuests table where the lastname is "Doe", and displays it on the page:

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);
```

```

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

$sql = "SELECT id, firstname, lastname FROM MyGuests WHERE lastname='Doe'";
$result = $conn->query($sql);

if ($result->num_rows > 0) {
    // output data of each row
    while($row = $result->fetch_assoc()) {
        echo "id: " . $row["id"]. " - Name: " . $row["firstname"]. " " . $row["lastname"]. "<br>";
    }
} else {
    echo "0 results";
}
$conn->close();
?>

```

Code lines to explain from the example above:

First, we set up the SQL query that selects the id, firstname and lastname columns from the MyGuests table where the lastname is "Doe". The next line of code runs the query and puts the resulting data into a variable called \$result.

Then, the `function num_rows()` checks if there are more than zero rows returned.

If there are more than zero rows returned, the function `fetch_assoc()` puts all the results into an associative array that we can loop through. The `while()` loop loops through the result set and outputs the data from the id, firstname and lastname columns.

Note: From above all examples we can understand one thing that for using database in php,

1. **first we need to create a database connection and after successful creation of database connection.** A connection is a computer science facility that allows the user to connect with the database server software. A user can connect with the database server, whether on the same machine or remote locations.
2. **we have put a query reference to a variable and query can be executed on database connection like given below**

```

$sql = "any data base query";
$conn->query($sql); // excution of query

```

3. **Query can be for create database, create table, insert data, delete data, update data, select data, select data with where clause**